

obj-prog
2.0

Sugeneruota Doxygen 1.17.0

1 Programos naudojimas:	1
1.1 Reikalavimai:	1
1.2 Programos diegimas:	1
1.3 Programos naudojimas:	1
1.4 v1.5:	1
1.4.1 Pridėta bazinė klasė Zmogus:	1
1.5 v1.2:	2
1.5.1 Realizuota "Rule of Five":	2
1.5.1.1 Įvesties operatoriaus naudojimas (veikia su visais įvedimo srautais):	2
1.5.1.2 Išvesties operatoriaus naudojimas (veikia su visais išvesties srautais):	2
1.5.2 Studento klasės metodų testas.	2
1.6 v1.1 testavimas:	2
1.6.1 Spartos palyginimas:	3
1.6.1.1 Pastebėjimai	3
1.7 Relizų aprašas	4
2 Direktorijų hierarchija	5
2.1 Direktorijos	5
3 Hierarchijos Indeksas	7
3.1 Klasių hierarchija	7
4 Klasės Indeksas	9
4.1 Klasės	9
5 Failo Indeksas	11
5.1 Failai	11
6 Direktorijų dokumentacija	13
6.1 vector Direktorijos aprašas	13
7 Klasės Dokumentacija	15
7.1 Studentas Klasė	15
7.1.1 Smulkus aprašymas	17
7.1.2 Konstruktoriaus ir Destruktoriaus Dokumentacija	18
7.1.2.1 Studentas() [1/5]	18
7.1.2.2 Studentas() [2/5]	18
7.1.2.3 Studentas() [3/5]	18
7.1.2.4 Studentas() [4/5]	18
7.1.2.5 Studentas() [5/5]	19
7.1.2.6 ~Studentas()	19
7.1.3 Metodų Dokumentacija	19
7.1.3.1 addNd()	19
7.1.3.2 egzaminoRezultatolvestis()	19

7.1.3.3 egzRandom()	19
7.1.3.4 galutinis()	19
7.1.3.5 genVardaPavarde()	20
7.1.3.6 getNd()	20
7.1.3.7 getRez()	20
7.1.3.8 namuDarbuRezultatulvestis()	20
7.1.3.9 ndRandom()	21
7.1.3.10 operator=() [1/2]	21
7.1.3.11 operator=() [2/2]	21
7.1.3.12 read()	21
7.1.3.13 readRandom()	22
7.1.3.14 readSemiRandom()	22
7.1.3.15 readStudentConsole()	22
7.1.3.16 readStudentStream()	22
7.1.3.17 resizeNd()	23
7.1.3.18 setNd()	23
7.1.3.19 setRez()	23
7.1.3.20 studentoVardoPavardesIvestis()	24
7.1.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija	24
7.1.4.1 operator<<	24
7.1.4.2 operator>>	24
7.1.5 Atributų Dokumentacija	24
7.1.5.1 nd	24
7.1.5.2 rez	25
7.2 Timer Klasė	25
7.2.1 Smulkus aprašymas	25
7.2.2 Tipo Aprašymo Dokumentacija	25
7.2.2.1 durationDouble	25
7.2.2.2 hrClock	26
7.2.3 Konstruktoriaus ir Destruktoriaus Dokumentacija	26
7.2.3.1 Timer()	26
7.2.4 Metodų Dokumentacija	26
7.2.4.1 elapsed()	26
7.2.4.2 reset()	26
7.2.5 Atributų Dokumentacija	26
7.2.5.1 start	26
7.3 Zmogus Klasė	27
7.3.1 Smulkus aprašymas	27
7.3.2 Konstruktoriaus ir Destruktoriaus Dokumentacija	28
7.3.2.1 Zmogus() [1/4]	28
7.3.2.2 Zmogus() [2/4]	28
7.3.2.3 Zmogus() [3/4]	28

7.3.2.4 Zmogus() [4/4]	28
7.3.2.5 ~Zmogus()	28
7.3.3 Metodų Dokumentacija	29
7.3.3.1 getPavarde()	29
7.3.3.2 getVardas()	29
7.3.3.3 operator=() [1/2]	29
7.3.3.4 operator=() [2/2]	29
7.3.3.5 setPavarde()	29
7.3.3.6 setVardas()	30
7.3.4 Atributų Dokumentacija	30
7.3.4.1 pavarde	30
7.3.4.2 vardas	30
8 Failo Dokumentacija	31
8.1 gtest.cpp Failo Nuoroda	31
8.1.1 Funkcijos Dokumentacija	31
8.1.1.1 TEST() [1/6]	31
8.1.1.2 TEST() [2/6]	31
8.1.1.3 TEST() [3/6]	32
8.1.1.4 TEST() [4/6]	32
8.1.1.5 TEST() [5/6]	32
8.1.1.6 TEST() [6/6]	32
8.2 gtest.cpp	32
8.3 README.md Failo Nuoroda	33
8.4 vector/apdorojimas.cpp Failo Nuoroda	33
8.4.1 Funkcijos Dokumentacija	34
8.4.1.1 failoGeneravimas()	34
8.4.1.2 setw()	34
8.5 apdorojimas.cpp	34
8.6 vector/apdorojimas.h Failo Nuoroda	35
8.6.1 Funkcijos Dokumentacija	35
8.6.1.1 failoGeneravimas()	35
8.6.1.2 setw()	36
8.7 apdorojimas.h	37
8.8 vector/apdorojimas.tpp Failo Nuoroda	37
8.8.1 Funkcijos Dokumentacija	38
8.8.1.1 rusiavimasPagal()	38
8.8.1.2 rusiavimasSkirstymas()	38
8.8.1.3 skirstymas()	39
8.8.1.4 skirstymasStrat1()	39
8.8.1.5 skirstymasStrat2()	40
8.8.1.6 skirstymasStrat3()	40

8.9 apdorojimas.tpp	41
8.10 vector/isvestis.cpp Failo Nuoroda	42
8.10.1 Funkcijos Dokumentacija	43
8.10.1.1 failoPasirinkimas()	43
8.10.1.2 failoUzklausa()	43
8.10.1.3 gautiTipoPasirinkima()	44
8.10.1.4 lietuviskosRaides()	44
8.10.1.5 medianosUzklausa()	44
8.10.1.6 menu()	44
8.10.1.7 ndPasirinkimas()	45
8.10.1.8 rusiavimoPasirinkimas()	45
8.10.1.9 strategijosPasirinkimas()	45
8.10.1.10 studentuPasirinkimas()	45
8.10.1.11 testavimoPasirinkimas()	46
8.11 isvestis.cpp	46
8.12 vector/isvestis.h Failo Nuoroda	48
8.12.1 Funkcijos Dokumentacija	48
8.12.1.1 failoPasirinkimas()	48
8.12.1.2 failoUzklausa()	49
8.12.1.3 gautiTipoPasirinkima()	49
8.12.1.4 lietuviskosRaides()	49
8.12.1.5 medianosUzklausa()	50
8.12.1.6 menu()	50
8.12.1.7 ndPasirinkimas()	50
8.12.1.8 rusiavimoPasirinkimas()	50
8.12.1.9 strategijosPasirinkimas()	51
8.12.1.10 studentuPasirinkimas()	51
8.12.1.11 testavimoPasirinkimas()	51
8.13 isvestis.h	52
8.14 vector/isvestis.tpp Failo Nuoroda	52
8.14.1 Funkcijos Dokumentacija	52
8.14.1.1 isvestis()	52
8.15 isvestis.tpp	54
8.16 vector/ivestis.cpp Failo Nuoroda	54
8.16.1 Funkcijos Dokumentacija	55
8.16.1.1 arTikSkaicius()	55
8.16.1.2 cinEOFgaudymas()	55
8.16.1.3 gautiPatvirtinima()	55
8.16.1.4 gautiSkaiciu()	56
8.16.1.5 ivestiStudentus()	56
8.17 ivestis.cpp	56
8.18 vector/ivestis.h Failo Nuoroda	58

8.18.1 Funkcijos Dokumentacija	58
8.18.1.1 arTikSkaicius()	58
8.18.1.2 cinEOFgaudymas()	59
8.18.1.3 egzaminoRezultatolvestis()	59
8.18.1.4 gautiPatvirtinima()	59
8.18.1.5 gautiSkaiciu()	59
8.18.1.6 ivestiStudentus()	60
8.18.1.7 namuDarbuRezultatulvestis()	60
8.18.1.8 skaitymas()	60
8.18.1.9 skaitymasIsFailo()	61
8.18.1.10 studentoVardoPavardesIvestis()	61
8.19 ivestis.h	62
8.20 vector/ivestis.tpp Failo Nuoroda	62
8.20.1 Funkcijos Dokumentacija	62
8.20.1.1 skaitymasIsFailo()	62
8.21 ivestis.tpp	63
8.22 vector/main.cpp Failo Nuoroda	63
8.22.1 Funkcijos Dokumentacija	64
8.22.1.1 failoApdorojimas()	64
8.22.1.2 main()	64
8.23 main.cpp	64
8.24 vector/main.h Failo Nuoroda	66
8.24.1 Funkcijos Dokumentacija	67
8.24.1.1 failoApdorojimas()	67
8.25 main.h	67
8.26 vector/main.tpp Failo Nuoroda	68
8.26.1 Funkcijos Dokumentacija	68
8.26.1.1 failoTestavimas()	68
8.26.1.2 skirstymoPasirinkimas() [1/2]	69
8.26.1.3 skirstymoPasirinkimas() [2/2]	69
8.27 main.tpp	70
8.28 vector/random.cpp Failo Nuoroda	71
8.28.1 Funkcijos Dokumentacija	71
8.28.1.1 ivestiStudentusRandom()	71
8.28.1.2 rng()	72
8.29 random.cpp	72
8.30 vector/random.h Failo Nuoroda	72
8.30.1 Funkcijos Dokumentacija	73
8.30.1.1 ivestiStudentusRandom()	73
8.30.1.2 rng()	73
8.31 random.h	73
8.32 vector/studentas.cpp Failo Nuoroda	74

8.32.1 Funkcijos Dokumentacija	74
8.32.1.1 operator<<()	74
8.32.1.2 operator>>()	74
8.33 studentas.cpp	74
8.34 vector/studentas.h Failo Nuoroda	78
8.34.1 Kintamojo Dokumentacija	78
8.34.1.1 maxNdKiekis	78
8.35 studentas.h	78
8.36 vector/test.cpp Failo Nuoroda	79
8.36.1 Funkcijos Dokumentacija	80
8.36.1.1 testas()	80
8.36.1.2 testCopyAssignmentOperator()	80
8.36.1.3 testCopyKonstruktorius()	80
8.36.1.4 testDefaultKonstruktorius()	81
8.36.1.5 testDestructor()	81
8.36.1.6 testInputOperator()	81
8.36.1.7 testMoveAssignmentOperator()	81
8.36.1.8 testMoveKonstruktorius()	81
8.36.1.9 testoRezultatas()	81
8.36.1.10 testOutputOperator()	82
8.36.1.11 testStreamKonstruktorius()	82
8.37 test.cpp	82
8.38 vector/test.h Failo Nuoroda	83
8.38.1 Funkcijos Dokumentacija	84
8.38.1.1 testas()	84
8.38.1.2 testCopyAssignmentOperator()	84
8.38.1.3 testCopyKonstruktorius()	84
8.38.1.4 testDefaultKonstruktorius()	85
8.38.1.5 testDestructor()	85
8.38.1.6 testInputOperator()	85
8.38.1.7 testMoveAssignmentOperator()	85
8.38.1.8 testMoveKonstruktorius()	85
8.38.1.9 testoRezultatas()	85
8.38.1.10 testOutputOperator()	86
8.38.1.11 testStreamKonstruktorius()	86
8.39 test.h	86
8.40 vector/Timer.h Failo Nuoroda	86
8.41 Timer.h	87
8.42 vector/zmogus.cpp Failo Nuoroda	87
8.43 zmogus.cpp	87
8.44 vector/zmogus.h Failo Nuoroda	88
8.45 zmogus.h	88

skyrius 1

Programos naudojimas:

1.1 Reikalavimai:

C++ kompiliatorius su C++20 palaikymu. Unix OS (Linux arba MacOS) arba WSL (Windows Subsystem for Linux)
git

1.2 Programos diegimas:

```
git clone https://github.com/AtlantasLTU/obj-prog-2.git cd ./obj-prog-2 make ./main
```

1.3 Programos naudojimas:

Funkcijos: Pasirinktinai išvestis į failą arba į terminalą. Pasirinktinai galutinio rezultato skaičiavimas, remiantis vidurkiu arba mediana. Direktoriijoje esančių .txt failų pasirinkimas. Rūšiavimas pasirinktu būdu.

1 parinktis - rankinis duomenų įvedimas, studento vardo, pavardės, namų darbų rezultatų, egzamino rezultato, jų apdorojimas ir išvedimas. 2 parinktis - pusiau rankinis duomenų įvedimas, studento vardo, pavardės, rezultatų generavimas, duomenų apdorojimas ir išvedimas. 3 parinktis - automatinis studentų vardų, pavardžių, rezultatų generavimas, jų apdorojimas ir išvedimas. 4 parinktis - skaitymas iš pasirinktino failo, rūšiavimas pasirinktinu būdu, duomenų apdorojimas ir išvedimas. 5 parinktis - testavimas su failais, pasirenkamas failas, konteinerio tipas, strategija, testų skaičius, failai apdorojami (nuskaitymas, skaičiavimas, rūšiavimas, skirstymas) ir išvedami testo rezultatai į terminalą. 6 parinktis - studentų failų generavimas, studentų, namų darbų kiekio pasirinkimas ir išvedimas į studentai*.txt failą. 7 parinktis - studento klasės testavimas. 8 parinktis - programos nutraukimas.

1.4 v1.5:

1.4.1 Pridėta bazinė klasė Zmogus:

Klasė studentas dabar išvestinė. Studento klasė paveldi iš žmogaus klasės vardą bei pavardę, metodus susijusiais su šiais kintamaisiais Klasė abstrakti dėl virtualaus destruktoriaus.

- Studento klasės rule of five atnaujintas, kad veiktų su bazinė klase.
- Rule of five taip pat realizuotas Zmogaus klasėje.
- Išlaikytas v1.2 testas bei funkcionalumas.

Bandant sukurti Zmogaus tipo objektą, išmeta klaidą, kad klasė abstrakti:

1.5 v1.2:

1.5.1 Realizuota "Rule of Five":

Destruktorius Copy konstruktorius Copy assignment operatorius Move konstruktorius Move assignment operatorius

- Realizuotas įvesties ir išvesties operatorių perdengimas.
- Sukurtas studento klasės metodų testas.

1.5.1.1 Įvesties operatoriaus naudojimas (veikia su visais įvedimo srautais):

Kai naudojamas `operator>>` su failo srautu, pažymių kiekis nustatomas automatiškai: visi skaičiai po vardo ir pavardės laikomi namų darbų pažymiais, o pats paskutinis – egzamino balu.

```
Studentas A;  
std::istringstream iss("Jonas Jonaitis 1 2 10");  
iss >> A; // įvestis iš įvedimo srauto.  
std::cin >> A; // įvestis iš konsolės.
```

1.5.1.2 Išvesties operatoriaus naudojimas (veikia su visais išvesties srautais):

`Studentas` A; `std::cout << A;` // išvestis į ekraną `std::ostringstream out; out << A;` // išvesties srauto kūrimas `std::ofstream fout("studentai.txt"); fout << A;` // išveda į failą

1.5.2 Studento klasės metodų testas.

1.6 v1.1 testavimas:

- Kompiuterio, su kuriuo testuota parametrai:

1.6.0.0.1 Skirstymo strategijos:

- 0 - pradiniam relize naudota strategija. Dviejų konteinerių "vargšiukų" ir "kietiakų" sukūrimas, duomenys perkeliama iš "studentai" konteinerio su `std::move`.
- 1 strategija: Bendro studentai konteinerio (vector, list ir deque tipų) skaidymas (rūšiavimas) į du naujus to paties tipo konteinerius: "vargšiukų" ir "kietiakų". Dviejų konteinerių "vargšiukų" ir "kietiakų" sukūrimas, duomenys kopijuojami iš "studentai" konteinerio.
- 2 strategija: Bendro studentų konteinerio (vector, list ir deque) skaidymas (rūšiavimas) panaudojant tik vieną naują konteinerį: "vargšiukai". Studentai konteineris išrūšiuotas, randamas iteratorius rodantis į pirmąjį galvočių, viskas iki iteratoriaus perkeliama į vargšiukų konteinerį ir ištrinama iš studentai konteinerio. Iteratoriui rasti naudojamas `lower_bound` metodas
- 3 strategija: Bendro studentų konteinerio (vector, list ir deque) skaidymas (rūšiavimas) panaudojant greičiausiai veikianti 1 arba 2 strategiją įtraukiant į ją "efektyvius" darbo su konteineriais metodus. Šioje strategijoje naudojamas `partition` metodas.

1.6.0.0.2 Atlikta programos veikimo greičio (spartos) analizė:

- lyginamas v1.0 struct tipas su v1.1 realizuota klasė.
- lyginamas vienas konteinerius - vektorius, su pačia greičiausia dalijimo strategija - 2.
- 100000 ir 1000000 studentų failai.
- lyginamos kompiliatoriaus optimizavimo vėliavėlės (-Ofast, -O1, -O2, -O3).

Testuota tik naudojant terminalą, visos kitos pašalinės programos testavimo metu buvo išjungtos bei įrenginys "performance" režime. Visi testavimo atvejai testuoti 10 kartų, su medianų skaičiavimu.

1.6.1 Spartos palyginimas:**1.6.1.0.1 100000 studentų:**

Tipas	Optimizavimo vėliavėlė	Bendras veikimo laikas (s)	Failo dydis (KB)
struct	-Ofast	1.65366 s	349.3 KB
class	-Ofast	4.66815 s	332.7 KB
struct	-O1	2.43243 s	300.6 KB
class	-O1	7.11753 s	293.2 KB
struct	-O2	2.24629 s	314.2 KB
class	-O2	6.3197 s	308.2 KB
struct	-O3	2.1662 s	349.5 KB
class	-O3	5.94076 s	332.7 KB

1.6.1.0.2 1000000 studentų:

Tipas	Optimizavimo vėliavėlė	Bendras veikimo laikas (s)	Failo dydis (KB)
struct	-Ofast	18.4434 s	349.3 KB
class	-Ofast	66.1051 s	332.7 KB
struct	-O1	20.5588 s	300.6 KB
class	-O1	79.2437 s	293.2 KB
struct	-O2	18.8914 s	314.2 KB
class	-O2	72.3371 s	308.2 KB
struct	-O3	18.0112 s	349.5 KB
class	-O3	55.4038 s	332.7 KB

1.6.1.1 Pastebėjimai

Ta pati programa perdaryta su class tipu lėtesnė ~3x negu su struct, priklausomai nuo optimizacijos vėliavėlės. Taip yra todėl, kad class tipo programoje, galutinis balas skaičiuojamas, kai jo prireikia ir nėra saugojamas atminty, tai sutaupo atminties, tačiau padidina skaičiavimų laiką. Pagrindė -Ofast vėliavėlė greičiausia, tačiau esant milijonui studentų -O3 aplenkia -Ofast vėliavėlę, galbūt -O3 geriau suoptimizuoja failų skaitymą. Mažiausią failą sukuria -O1 vėliavėlė, o didžiausią -O3. Taip yra todėl, kad -O1 skirtas balansui tarp greičio ir dydžio, o -O3 skirtas našesniai programos veikimui.

Nors struct tipas greitesnis, tačiau class tipas saugesnis ir plečiant programos apimtį, enkapsuliacija, paveldamumas ir kitos class tipo ypatybės padaro jį naudingesniu.

1.7 Relizų aprašas

v1.2

Realizuota rule of five.
Realizuotas įvesties išvesties operatorių perkrovimas.
Sukurtas studento klasės metodų testas.

v1.1

Studento struktūra paversta į klasę
Implementuoti get'eriai, set'eriai, konstruktoriai bei destruktorius.
Kintamieji privatūs, pasiekiami tik per studento klasės metodus.
Refaktorintas ir patobulintas random vardų, pavardžių generatorius.
Nebesaugojamas kintamasis galutinio rezultato, jis skaičiuojamas tik, kai jo prireikia.

v1.0

Padidintas template naudojimas, siekiant ištestuoti vector, deque ir list konteinerius.
Pridėtos trys studentų skirstymo į „vargšiukus“ (vidurkis < 5.0) ir „kietiakus“ strategijos:
1 strategija: Bendro konteinerio skaidymas į du naujus, kopijuojant į vargšukus ir galvočiai konteinerius.
2 strategija: „Vargšiukų“ perkėlimas į naują konteinerį, juos ištrinant iš studentų konteinerio.
3 strategija: Optimizuotas skirstymas naudojant efektyvius algoritmus (std::partition).

Paruoštas pilnas README.md su tyrimo rezultatais, lentelėmis ir naudojimo instrukcija.
Pridėtas Makefile lengvam programos kompiliavimui.

v0.4

Patobulinta išvestis.
Pridėtas skirstymas į "galvočius" ir "vargšiukus".
Pridėtas failų generavimas.
Pridėtas failų pasirinkimas.

v0.3

Kodas išskirstytas į daugiau dalių (.h ir .cpp failus).
Įterptas try-catch blokas failų egzistavimo tikrinimui, kitų klaidų gaudymui.
Panaudotas template rūšiavimo funkcijoje, kodo skaitomumui pagerinti.
Refaktorintas metu apšvarintas main.cpp failas.

v0.2

Pridėtas duomenų nuskaitymas iš išorinių failų.
Pridėti testavimo atvejai.

v0.1

Pridėtas automatinis pažymių generavimas.
Įdiegtas „sąžiningas“ vidurkio skaičiavimas (atsižvelgiant į trūkstantus namų darbus).
Patobulintas įvesties valdymas (apsauga nuo neteisingos įvesties).

V.pradinė

Sukurta studento struktūra.
Realizuotas vidurkio ir medianos skaičiavimas.
Pradinė įvesties apsauga.
Hardcoded reikšmės testavimui.

skyrius 2

Direktorių hierarchija

2.1 Direktorijos

vector	13
apdorojimas.cpp	33
apdorojimas.h	35
apdorojimas.hpp	37
isvestis.cpp	42
isvestis.h	48
isvestis.hpp	52
investis.cpp	54
investis.h	58
investis.hpp	62
main.cpp	63
main.h	66
main.hpp	68
random.cpp	71
random.h	72
studentas.cpp	74
studentas.h	78
test.cpp	79
test.h	83
Timer.h	86
zmogus.cpp	87
zmogus.h	88

skyrius 3

Hierarchijos Indeksas

3.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

Timer	25
Zmogus	27
Studentas	15

skyrius 4

Klasės Indeksas

4.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

Studentas		
	Klasė, sukurianti studento objektą su pažymiais	15
Timer		25
Zmogus		
	Abstrakti bazinė klasė, turinti vardą ir pavardę	27

skyrius 5

Failo Indeksas

5.1 Failai

Visų failų sąrašas su trumpais aprašymais:

gtest.cpp	31
vector/apdorojimas.cpp	33
vector/apdorojimas.h	35
vector/apdorojimas.tpp	37
vector/isvestis.cpp	42
vector/isvestis.h	48
vector/isvestis.tpp	52
vector/ivestis.cpp	54
vector/ivestis.h	58
vector/ivestis.tpp	62
vector/main.cpp	63
vector/main.h	66
vector/main.tpp	68
vector/random.cpp	71
vector/random.h	72
vector/studentas.cpp	74
vector/studentas.h	78
vector/test.cpp	79
vector/test.h	83
vector/Timer.h	86
vector/zmogus.cpp	87
vector/zmogus.h	88

skyrius 6

Direktorių dokumentacija

6.1 vector Directorijos aprašas

Failai

- failas [apdorojimas.cpp](#)
- failas [apdorojimas.h](#)
- failas [apdorojimas.tpp](#)
- failas [isvestis.cpp](#)
- failas [isvestis.h](#)
- failas [isvestis.tpp](#)
- failas [investis.cpp](#)
- failas [investis.h](#)
- failas [investis.tpp](#)
- failas [main.cpp](#)
- failas [main.h](#)
- failas [main.tpp](#)
- failas [random.cpp](#)
- failas [random.h](#)
- failas [studentas.cpp](#)
- failas [studentas.h](#)
- failas [test.cpp](#)
- failas [test.h](#)
- failas [Timer.h](#)
- failas [zmogus.cpp](#)
- failas [zmogus.h](#)

skyrius 7

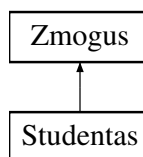
Klasės Dokumentacija

7.1 Studentas Klasė

Klasė, sukurianti studento objektą su pažymiais.

```
#include <studentas.h>
```

Paveldimumo diagrama Studentas:



Vieši Metodai

- `Studentas ()`
Default konstruktorius.
- `Studentas (std::string v, std::string p, std::vector< int > n, int r)`
Konstruktorius su visais parametrais.
- `Studentas (std::istream &is, int ndKiekis=0)`
Konstruktorius, nuskaitantis studentą iš srauto.
- `Studentas (const Studentas &kitas)`
Kopijavimo konstruktorius.
- `Studentas & operator= (const Studentas &)`
Kopijavimo priskyrimo operatorius.
- `Studentas (Studentas &&) noexcept`
Perkėlimo konstruktorius.
- `Studentas & operator= (Studentas &&) noexcept`
Perkėlimo priskyrimo operatorius.
- `const std::vector< int > & getNd () const`
- `int getRez () const`
- `void setNd (std::vector< int > n)`
Nustato namų darbų pažymius.
- `void setRez (int r)`

- *Nustato egzamino rezultatą.*
- `std::istream & readStudentStream (std::istream &is, int ndKiekis)`
Nuskaityto studentą iš failo srauto.
- `std::istream & readStudentConsole (std::istream &is)`
Nuskaityto studentą iš terminalo.
- `void read (std::istream &is, int ndKiekis=0)`
Nuskaityto studentą iš srauto (automatiškai parenka readStudentConsole arba readStudentStream).
- `bool readSemiRandom ()`
Dalinai atsitiktinis įvedimas: vardas/pavardė ranka, pažymiai atsitiktiniai.
- `void readRandom (int ndKiekis)`
Visiškai atsitiktinis studento ir jo rezultatų sugeneravimas.
- `void addNd (int paz)`
Prideda vieną namų darbų pažymį.
- `void resizeNd (int n, int skaicius=0)`
Resize'ina namų darbų vektorių ir užpildo naujas vietas nurodyta reikšme.
- `double galutinis (bool medianos, int ndKiekis=0) const`
Apskaičiuoja galutinį pažymį.
- `~Studentas ()`
Destruktorius.

Vieši Metodai inherited from **Zmogus**

- `Zmogus ()`
Numatytasis konstruktorius – tuščias vardas ir pavardė.
- `Zmogus (const std::string v, const std::string p)`
Konstruktorius su vardu ir pavarde.
- `Zmogus (const Zmogus &kitas)`
Kopijavimo konstruktorius.
- `Zmogus & operator= (const Zmogus &)`
Kopijavimo priskyrimo operatorius.
- `Zmogus (Zmogus &&) noexcept`
Perkėlimo konstruktorius.
- `Zmogus & operator= (Zmogus &&) noexcept`
Perkėlimo priskyrimo operatorius.
- `const std::string & getVardas () const`
- `const std::string & getPavarde () const`
- `void setVardas (std::string v)`
Nustato vardą.
- `void setPavarde (std::string p)`
Nustato pavardę.
- `virtual ~Zmogus ()=0`
Virtualus destruktoriaus – padaro klasę abstrakčia.

Privatatūs Metodai

- bool `studentoVardoPavardesIvestis` (const std::string &eilute)
Nuskaityto vardą ir pavardę iš eilutės.
- void `namuDarbuRezultatulvestis` ()
Nuskaityto namų darbų rezultatus iš terminalo.
- void `egzaminoRezultatulvestis` ()
Nuskaityto egzamino rezultatą iš terminalo.
- void `genVardaPavarde` ()
Atsitiktinai sugeneruoja vardą ir pavardę.
- void `ndRandom` (int ndKiekis)
Atsitiktinai sugeneruoja namų darbų rezultatus.
- void `egzRandom` ()
Atsitiktinai sugeneruoja egzamino rezultatą.

Privatūs Atributai

- std::vector< int > `nd`
Namų darbų pažymių vektorius.
- int `rez`
Egzamino pažymys.

Draugai

- std::istream & `operator>>` (std::istream &in, `Studentas` &A)
Ivesties operatorius (>>).
- std::ostream & `operator<<` (std::ostream &out, const `Studentas` &A)
Išvesties operatorius (<<).

Additional Inherited Members

Apsaugoti Atributai inherited from `Zmogus`

- std::string `vardas`
Zmogaus vardas.
- std::string `pavarde`
Zmogaus pavardė.

7.1.1 Smulkus aprašymas

Klasė, sukurianti studento objektą su pažymiais.

Paveldi iš klasės `Zmogus`. Saugo namų darbų pažymius ir egzamino rezultatą. Leidžia skaičiuoti galutinį pažymį (pagal vidurkį arba medianą).

Apibrėžimas failo `studentas.h` eilutėje 19.

7.1.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

7.1.2.1 Studentas() [1/5]

```
Studentas::Studentas () [inline]
```

Default konstruktorius.

Apibrėžimas failo [studentas.h](#) eilutėje 51.

7.1.2.2 Studentas() [2/5]

```
Studentas::Studentas (
    std::string v,
    std::string p,
    std::vector< int > n,
    int r) [inline]
```

Konstruktorius su visais parametrais.

Parametrai

<i>v</i>	Vardas.
<i>p</i>	Pavardė.
<i>n</i>	Namų darbų pažymių vektorius.
<i>r</i>	Egzamino rezultatas.

Apibrėžimas failo [studentas.h](#) eilutėje 60.

7.1.2.3 Studentas() [3/5]

```
Studentas::Studentas (
    std::istream & is,
    int ndKiekis = 0) [inline]
```

Konstruktorius, nuskaitantis studentą iš srauto.

Parametrai

<i>is</i>	Išvesties srautas.
<i>ndKiekis</i>	Namų darbų skaičius (jei 0, tai nustatoma iš duomenų).

Apibrėžimas failo [studentas.h](#) eilutėje 68.

7.1.2.4 Studentas() [4/5]

```
Studentas::Studentas (
    const Studentas & kitas)
```

Kopijavimo konstruktorius.

Apibrėžimas failo [studentas.cpp](#) eilutėje 222.

7.1.2.5 Studentas() [5/5]

```
Studentas::Studentas (
    Studentas && kitas) [noexcept]
```

Perkėlimo konstruktorius.

Apibrėžimas failo [studentas.cpp](#) eilutėje 239.

7.1.2.6 ~Studentas()

```
Studentas::~~Studentas ()
```

Destruktorius.

Apibrėžimas failo [studentas.cpp](#) eilutėje 216.

7.1.3 Metodų Dokumentacija**7.1.3.1 addNd()**

```
void Studentas::addNd (
    int paz)
```

Prideda vieną namų darbų pažymį.

Parametrai

<i>paz</i>	Pažymys (1-10).
------------	-----------------

Apibrėžimas failo [studentas.cpp](#) eilutėje 12.

7.1.3.2 egzaminoRezultatIvestis()

```
void Studentas::egzaminoRezultatoIvestis () [private]
```

Nuskaityto egzamino rezultatą iš terminalo.

Apibrėžimas failo [studentas.cpp](#) eilutėje 144.

7.1.3.3 egzRandom()

```
void Studentas::egzRandom () [private]
```

Atsitiktinai sugeneruoja egzamino rezultatą.

Apibrėžimas failo [studentas.cpp](#) eilutėje 210.

7.1.3.4 galutinis()

```
double Studentas::galutinis (
    bool medianos,
    int ndKiekis = 0) const
```

Apskaičiuoja galutinį pažymį.

Parametrai

<i>medianos</i>	Jei true – naudoja medianą, kitu atveju – vidurkį.
-----------------	--

<i>ndKiekis</i>	Jei >0 – naudoja šį ND skaičių, kitu atveju – maxNdKiekis .
-----------------	--

Gražina

Galutinis pažymys ($0.4 * \text{ND} + 0.6 * \text{egz.}$).

Apibrėžimas failo [studentas.cpp](#) eilutėje 22.

7.1.3.5 genVardaPavarde()

```
void Studentas::genVardaPavarde () [private]
```

Atsitiktinai sugeneruoja vardą ir pavardę.

Apibrėžimas failo [studentas.cpp](#) eilutėje 181.

7.1.3.6 getNd()

```
const std::vector< int > & Studentas::getNd () const [inline]
```

Gražina

Namų darbų pažymių vektoriaus konstantinė nuoroda.

Apibrėžimas failo [studentas.h](#) eilutėje 84.

7.1.3.7 getRez()

```
int Studentas::getRez () const [inline]
```

Gražina

Egzamino rezultatas.

Apibrėžimas failo [studentas.h](#) eilutėje 87.

7.1.3.8 namuDarbuRezultatulvestis()

```
void Studentas::namuDarbuRezultatuIvestis () [private]
```

Nuskaito namų darbų rezultatus iš terminalo.

Apibrėžimas failo [studentas.cpp](#) eilutėje 128.

7.1.3.9 ndRandom()

```
void Studentas::ndRandom (
    int ndKiekis) [private]
```

Atsitiktinai sugeneruoja namų darbų rezultatus.

Parametrai

<i>ndKiekis</i>	Kiek rezultatų generuoti.
-----------------	---------------------------

Apibrėžimas failo [studentas.cpp](#) eilutėje 203.

7.1.3.10 operator=() [1/2]

```
Studentas & Studentas::operator= (
    const Studentas & kitas)
```

Kopijavimo priskyrimo operatorius.

Apibrėžimas failo [studentas.cpp](#) eilutėje 228.

7.1.3.11 operator=() [2/2]

```
Studentas & Studentas::operator= (
    Studentas && kitas) [noexcept]
```

Perkėlimo priskyrimo operatorius.

Apibrėžimas failo [studentas.cpp](#) eilutėje 245.

7.1.3.12 read()

```
void Studentas::read (
    std::istream & is,
    int ndKiekis = 0)
```

Nuskaito studentą iš srauto (automatiškai parenka readStudentConsole arba readStudentStream).

Parametrai

<i>is</i>	Ivesties srautas.
<i>ndKiekis</i>	Namų darbų skaičius (naudojamas tik failo skaitymui).

Apibrėžimas failo [studentas.cpp](#) eilutėje 268.

7.1.3.13 readRandom()

```
void Studentas::readRandom (
    int ndKiekis)
```

Visiškai atsitiktinis studento ir jo rezultatų sugeneravimas.

Parametrai

<i>ndKiekis</i>	Kiek namų darbų pažymių sugeneruoti.
-----------------	--------------------------------------

Apibrėžimas failo [studentas.cpp](#) eilutėje 174.

7.1.3.14 readSemiRandom()

```
bool Studentas::readSemiRandom ()
```

Dalinai atsitiktinis įvedimas: vardas/pavardė ranka, pažymiai atsitiktiniai.

Gražina

true jei vartotojas įvedė ne tuščią eilutę, false jei įveda ENTER - užbaigia įvedimą.

Apibrėžimas failo [studentas.cpp](#) eilutėje 149.

7.1.3.15 readStudentConsole()

```
std::istream & Studentas::readStudentConsole (
    std::istream & is)
```

Nuskaito studentą iš terminalo.

Parametrai

<i>is</i>	Įvesties srautas (paprastai std::cin).
-----------	--

Gražina

Srauto nuoroda.

Apibrėžimas failo [studentas.cpp](#) eilutėje 85.

7.1.3.16 readStudentStream()

```
std::istream & Studentas::readStudentStream (
    std::istream & is,
    int ndKiekis)
```

Nuskaito studentą iš failo srauto.

Parametrai

<i>is</i>	Įvesties srautas.
-----------	-------------------

<i>ndKiekis</i>	Jei >0, skaitoma būtent tiek nd rezultatų, kitaip skaitoma iki failo galo ir egzamino rezultatas nustatomas automatiškai.
-----------------	---

Gražina

Srauto nuoroda.

Apibrėžimas failo [studentas.cpp](#) eilutėje 53.

7.1.3.17 `resizeNd()`

```
void Studentas::resizeNd (
    int n,
    int skaicius = 0)
```

Resize'ina namų darbų vektorių ir užpildo naujas vietas nurodyta reikšme.

Parametrai

<i>n</i>	Naujas dydis.
<i>skaicius</i>	Reikšmė naujiems elementams (numatyta 0).

Apibrėžimas failo [studentas.cpp](#) eilutėje 17.

7.1.3.18 `setNd()`

```
void Studentas::setNd (
    std::vector< int > n) [inline]
```

Nustato namų darbų pažymius.

Parametrai

<i>n</i>	Naujas pažymių vektorius.
----------	---------------------------

Apibrėžimas failo [studentas.h](#) eilutėje 94.

7.1.3.19 `setRez()`

```
void Studentas::setRez (
    int r) [inline]
```

Nustato egzamino rezultatą.

Parametrai

<i>r</i>	Naujas egzamino pažymys.
----------	--------------------------

Apibrėžimas failo [studentas.h](#) eilutėje 100.

7.1.3.20 studentoVardoPavardesIvestis()

```
bool Studentas::studentoVardoPavardesIvestis (
    const std::string & eilute) [private]
```

Nuskaityto vardą ir pavardę iš eilutės.

Parametrai

<i>eilute</i>	Eilutė, kurioje tikimasi dviejų žodžių.
---------------	---

Gražina

true jei nuskaitymas sėkmingas, false jei formatas neteisingas.

Apibrėžimas failo [studentas.cpp](#) eilutėje 110.

7.1.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija

7.1.4.1 operator<<

```
std::ostream & operator<< (
    std::ostream & out,
    const Studentas & A) [friend]
```

Išvesties operatorius (<<).

Apibrėžimas failo [studentas.cpp](#) eilutėje 261.

7.1.4.2 operator>>

```
std::istream & operator>> (
    std::istream & in,
    Studentas & A) [friend]
```

Išvesties operatorius (>>).

Apibrėžimas failo [studentas.cpp](#) eilutėje 255.

7.1.5 Atributų Dokumentacija

7.1.5.1 nd

```
std::vector<int> Studentas::nd [private]
```

Namų darbų pažymių vektorius.

Apibrėžimas failo [studentas.h](#) eilutėje 20.

7.1.5.2 rez

```
int Studentas::rez [private]
```

Egzamino pažymys.

Apibrėžimas failo [studentas.h](#) eilutėje 21.

Dokumentacija šiai klasei sugeneruota iš šių failų:

- vector/[studentas.h](#)
- vector/[studentas.cpp](#)

7.2 Timer Klasė

```
#include <Timer.h>
```

Vieši Metodai

- [Timer](#) ()
- void [reset](#) ()
- double [elapsed](#) () const

Privatūs Tipai

- using [hrClock](#) = std::chrono::high_resolution_clock
- using [durationDouble](#) = std::chrono::duration<double>

Privatūs Atributai

- std::chrono::time_point< [hrClock](#) > [start](#)

7.2.1 Smulkus aprašymas

Apibrėžimas failo [Timer.h](#) eilutėje 2.

7.2.2 Tipo Aprašymo Dokumentacija

7.2.2.1 durationDouble

```
using Timer::durationDouble = std::chrono::duration<double> [private]
```

Apibrėžimas failo [Timer.h](#) eilutėje 6.

7.2.2.2 hrClock

```
using Timer::hrClock = std::chrono::high_resolution_clock [private]
```

Apibrėžimas failo [Timer.h](#) eilutėje 5.

7.2.3 Konstruktoriaus ir Destruktoriaus Dokumentacija

7.2.3.1 Timer()

```
Timer::Timer () [inline]
```

Apibrėžimas failo [Timer.h](#) eilutėje 9.

7.2.4 Metodų Dokumentacija

7.2.4.1 elapsed()

```
double Timer::elapsed () const [inline]
```

Apibrėžimas failo [Timer.h](#) eilutėje 13.

7.2.4.2 reset()

```
void Timer::reset () [inline]
```

Apibrėžimas failo [Timer.h](#) eilutėje 10.

7.2.5 Atributų Dokumentacija

7.2.5.1 start

```
std::chrono::time_point<hrClock> Timer::start [private]
```

Apibrėžimas failo [Timer.h](#) eilutėje 7.

Dokumentacija šiai klasei sugeneruota iš šio failo:

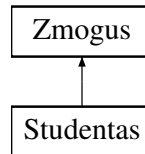
- [vector/Timer.h](#)

7.3 Zmogus Klasė

Abstrakti bazinė klasė, turinti vardą ir pavardę.

```
#include <zmogus.h>
```

Paveldimumo diagrama Zmogus:



Vieši Metodai

- `Zmogus ()`
Numatytasis konstruktorius – tuščias vardas ir pavardė.
- `Zmogus (const std::string v, const std::string p)`
Konstruktorius su vardu ir pavarde.
- `Zmogus (const Zmogus &kitas)`
Kopijavimo konstruktorius.
- `Zmogus & operator= (const Zmogus &)`
Kopijavimo priskyrimo operatorius.
- `Zmogus (Zmogus &&) noexcept`
Perkėlimo konstruktorius.
- `Zmogus & operator= (Zmogus &&) noexcept`
Perkėlimo priskyrimo operatorius.
- `const std::string & getVardas () const`
- `const std::string & getPavarde () const`
- `void setVardas (std::string v)`
Nustato vardą.
- `void setPavarde (std::string p)`
Nustato pavardę.
- `virtual ~Zmogus ()=0`
Virtualus destruktorius – padaro klasę abstrakčia.

Apsaugoti Atributai

- `std::string vardas`
Zmogaus vardas.
- `std::string pavarde`
Zmogaus pavardė.

7.3.1 Smulkus aprašymas

Abstrakti bazinė klasė, turinti vardą ir pavardę.

Klasė yra abstrakti dėl virtualaus destruktoriaus. Skirta paveldėjimui (pvz., `Studentas`).

Apibrėžimas failo `zmogus.h` eilutėje 13.

7.3.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

7.3.2.1 Zmogus() [1/4]

```
Zmogus::Zmogus () [inline]
```

Numatytasis konstruktorius – tuščias vardas ir pavardė.

Apibrėžimas failo [zmogus.h](#) eilutėje 20.

7.3.2.2 Zmogus() [2/4]

```
Zmogus::Zmogus (  
    const std::string v,  
    const std::string p) [inline]
```

Konstruktorius su vardu ir pavarde.

Parametrai

<i>v</i>	Vardas.
<i>p</i>	Pavardė.

Apibrėžimas failo [zmogus.h](#) eilutėje 27.

7.3.2.3 Zmogus() [3/4]

```
Zmogus::Zmogus (  
    const Zmogus & kitas)
```

Kopijavimo konstruktorius.

Apibrėžimas failo [zmogus.cpp](#) eilutėje 4.

7.3.2.4 Zmogus() [4/4]

```
Zmogus::Zmogus (  
    Zmogus && kitas) [noexcept]
```

Perkėlimo konstruktorius.

Apibrėžimas failo [zmogus.cpp](#) eilutėje 21.

7.3.2.5 ~Zmogus()

```
Zmogus::~Zmogus () [pure virtual]
```

Virtualus destruktoriaus – padaro klasę abstrakčia.

Apibrėžimas failo [zmogus.cpp](#) eilutėje 37.

7.3.3 Metodų Dokumentacija

7.3.3.1 getPavarde()

```
const std::string & Zmogus::getPavarde () const [inline]
```

Gražina

Pavarde (konstantinė nuoroda).

Apibrėžimas failo [zmogus.h](#) eilutėje 46.

7.3.3.2 getVardas()

```
const std::string & Zmogus::getVardas () const [inline]
```

Gražina

Vardą (konstantinė nuoroda).

Apibrėžimas failo [zmogus.h](#) eilutėje 43.

7.3.3.3 operator=() [1/2]

```
Zmogus & Zmogus::operator= (  
    const Zmogus & kitas)
```

Kopijavimo priskyrimo operatorius.

Apibrėžimas failo [zmogus.cpp](#) eilutėje 10.

7.3.3.4 operator=() [2/2]

```
Zmogus & Zmogus::operator= (  
    Zmogus && kitas) [noexcept]
```

Perkėlimo priskyrimo operatorius.

Apibrėžimas failo [zmogus.cpp](#) eilutėje 27.

7.3.3.5 setPavarde()

```
void Zmogus::setPavarde (  
    std::string p) [inline]
```

Nustato pavardę.

Parametrai

<i>p</i>	Nauja pavardė.
----------	----------------

Apibrėžimas failo [zmogus.h](#) eilutėje 58.

7.3.3.6 setVardas()

```
void Zmogus::setVardas (  
    std::string v)    [inline]
```

Nustato vardą.

Parametrai

v	Naujas vardas.
---	----------------

Apibrėžimas failo [zmogus.h](#) eilutėje 52.

7.3.4 Atributų Dokumentacija

7.3.4.1 pavarde

```
std::string Zmogus::pavarde    [protected]
```

Zmogaus pavardė.

Apibrėžimas failo [zmogus.h](#) eilutėje 16.

7.3.4.2 vardas

```
std::string Zmogus::vardas    [protected]
```

Zmogaus vardas.

Apibrėžimas failo [zmogus.h](#) eilutėje 15.

Dokumentacija šiai klasei sugeneruota iš šių failų:

- [vector/zmogus.h](#)
- [vector/zmogus.cpp](#)

skyrius 8

Failo Dokumentacija

8.1 gtest.cpp Failo Nuoroda

```
#include <gtest/gtest.h>
#include "../vector/studentas.h"
#include <sstream>
#include <memory>
```

Funkcijos

- [TEST](#) (RuleOfFive, CopyConstructor)
- [TEST](#) (RuleOfFive, CopyAssignment)
- [TEST](#) (RuleOfFive, MoveConstructor)
- [TEST](#) (RuleOfFive, MoveAssignment)
- [TEST](#) (RuleOfFive, Destructor)
- [TEST](#) (Functional, MedianCalculation)

8.1.1 Funkcijos Dokumentacija

8.1.1.1 TEST() [1/6]

```
TEST (
    Functional ,
    MedianCalculation )
```

Apibrėžimas failo [gtest.cpp](#) eilutėje 76.

8.1.1.2 TEST() [2/6]

```
TEST (
    RuleOfFive ,
    CopyAssignment )
```

Apibrėžimas failo [gtest.cpp](#) eilutėje 19.

8.1.1.3 TEST() [3/6]

```
TEST (
    RuleOfFive ,
    CopyConstructor )
```

Apibrėžimas failo [gtest.cpp](#) eilutėje 7.

8.1.1.4 TEST() [4/6]

```
TEST (
    RuleOfFive ,
    Destructor )
```

Apibrėžimas failo [gtest.cpp](#) eilutėje 65.

8.1.1.5 TEST() [5/6]

```
TEST (
    RuleOfFive ,
    MoveAssignment )
```

Apibrėžimas failo [gtest.cpp](#) eilutėje 48.

8.1.1.6 TEST() [6/6]

```
TEST (
    RuleOfFive ,
    MoveConstructor )
```

Apibrėžimas failo [gtest.cpp](#) eilutėje 32.

8.2 gtest.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include <gtest/gtest.h>
00002 #include "../vector/studentas.h"
00003 #include <sstream>
00004 #include <memory>
00005
00006 // 1. Kopijavimo konstruktorius
00007 TEST(RuleOfFive, CopyConstructor)
00008 {
00009     Studentas A("A", "AAA", {1, 2}, 10);
00010     Studentas B(A);
00011
00012     EXPECT_EQ(B.getVardas(), "A");
00013     EXPECT_EQ(B.getPavarde(), "AAA");
00014     EXPECT_EQ(B.getNd().size(), 2);
00015     EXPECT_EQ(B.getRez(), 10);
00016 }
00017
00018 // 2. Kopijavimo priskyrimas
00019 TEST(RuleOfFive, CopyAssignment)
00020 {
00021     Studentas A("A", "AAA", {1, 2}, 10);
00022     Studentas B;
```

```

00023     B = A;
00024
00025     EXPECT_EQ(B.getVardas(), "A");
00026     EXPECT_EQ(B.getPavarde(), "AAA");
00027     EXPECT_EQ(B.getNd().size(), 2);
00028     EXPECT_EQ(B.getRez(), 10);
00029 }
00030
00031 // 3. Perkëlimo konstruktorius
00032 TEST(RuleOfFive, MoveConstructor)
00033 {
00034     Studentas A("A", "AAA", {1, 2}, 10);
00035     Studentas B(std::move(A));
00036
00037     EXPECT_EQ(B.getVardas(), "A");
00038     EXPECT_EQ(B.getPavarde(), "AAA");
00039     EXPECT_EQ(B.getNd().size(), 2);
00040     EXPECT_EQ(B.getRez(), 10);
00041     EXPECT_TRUE(A.getVardas().empty());
00042     EXPECT_TRUE(A.getPavarde().empty());
00043     EXPECT_TRUE(A.getNd().empty());
00044     EXPECT_EQ(A.getRez(), 0);
00045 }
00046
00047 // 4. Perkëlimo priskyrimas
00048 TEST(RuleOfFive, MoveAssignment)
00049 {
00050     Studentas A("A", "AAA", {1, 2}, 10);
00051     Studentas B;
00052     B = std::move(A);
00053
00054     EXPECT_EQ(B.getVardas(), "A");
00055     EXPECT_EQ(B.getPavarde(), "AAA");
00056     EXPECT_EQ(B.getNd().size(), 2);
00057     EXPECT_EQ(B.getRez(), 10);
00058     EXPECT_TRUE(A.getVardas().empty());
00059     EXPECT_TRUE(A.getPavarde().empty());
00060     EXPECT_TRUE(A.getNd().empty());
00061     EXPECT_EQ(A.getRez(), 0);
00062 }
00063
00064 // 5. Destruktorius
00065 TEST(RuleOfFive, Destructor)
00066 {
00067     std::weak_ptr<Studentas> weak;
00068     {
00069         std::shared_ptr<Studentas> A = std::make_shared<Studentas>("A", "AAA", std::vector<int>{1,2},
10);
00070         weak = A;
00071     }
00072     EXPECT_TRUE(weak.expired());
00073 }
00074
00075 // 6. Papildomas: galutinis() su mediana
00076 TEST(Functional, MedianCalculation)
00077 {
00078     Studentas A("A", "AAA", {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}, 10);
00079     double result = A.galutinis(true, 10);
00080     EXPECT_DOUBLE_EQ(result, 8.2);
00081 }

```

8.3 README.md Failo Nuoroda

8.4 vector/apdorojimas.cpp Failo Nuoroda

```
#include "apdorojimas.h"
```

Funkcijos

- void **failoGeneravimas** (int studentuKiekis, int ndKiekis)
Generuoja failą su studentų duomenimis.
- void **setw** (const std::string &tekstas, int plotis, std::string &out)
Prideda tekstą prie išvesties eilutės, sulygiuodama jį nurodytu pločiu.

8.4.1 Funkcijos Dokumentacija

8.4.1.1 failoGeneravimas()

```
void failoGeneravimas (
    int studentuKiekis,
    int ndKiekis)
```

Generuoja failą su studentų duomenimis.

Parametrai

<i>studentuKiekis</i>	Kiek studentų generuoti.
<i>ndKiekis</i>	Kiek namų darbų rezultatų generuoti kiekvienam studentui.

Apibrėžimas failo [apdorojimas.cpp](#) eilutėje 3.

8.4.1.2 setw()

```
void setw (
    const std::string & tekstas,
    int plotis,
    std::string & out)
```

Prideda tekstą prie išvesties eilutės, sulygiuodama jį nurodytu pločiu.

Parametrai

<i>tekstas</i>	Tekstas
<i>plotis</i>	Minimalus eilutės plotis.
<i>out</i>	Nuoroda į išvesties eilutę.

Apibrėžimas failo [apdorojimas.cpp](#) eilutėje 29.

8.5 apdorojimas.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "apdorojimas.h"
00002
00003 void failoGeneravimas(int studentuKiekis, int ndKiekis)
00004 {
00005     std::string out;
00006     out.reserve(studentuKiekis*100/* 46+(ndKiekis*10)+studentuKiekis*(43+ndKiekis*10) */);
00007     setw("Vardas", 20, out);
00008     setw("Pavardė", 21, out);
00009     for(int i = 1; i <= ndKiekis; i++){
00010         setw(std::format("ND{} ", i), 10, out);
00011     }
00012     out += "Egz.\n";
00013     for(int i = 0; i < studentuKiekis; i++){
00014         Studentas A;
00015         A.readRandom(ndKiekis);
00016         setw(A.getVardas(), 20, out);
00017         setw(A.getPavarde(), 20, out);
00018         for(int j = 0; j < ndKiekis; j++){
```

```

00019         setw(std::to_string(A.getNd().at(j)), 10, out);
00020     }
00021     out += std::to_string(A.getRez()) + "\n";
00022 }
00023 std::string failoPavadinimas = std::string("studentai") + std::to_string(studentuKiekis) + ".txt";
00024 std::ofstream fout(failoPavadinimas);
00025 fout << out;
00026 fout.close();
00027 }
00028
00029 void setw(const std::string &tekstas, int plotis, std::string& out) {
00030     out += tekstas;
00031     if (tekstas.size() < plotis)
00032         out.append(plotis - tekstas.size(), ' ');
00033 };

```

8.6 vector/apdorojimas.h Failo Nuoroda

```

#include <algorithm>
#include <format>
#include "studentas.h"
#include "random.h"
#include <fstream>
#include <list>

```

Funkcijos

- void [failoGeneravimas](#) (int studentuKiekis, int ndKiekis)
Generuoja failą su studentų duomenimis.
- void [setw](#) (const std::string &tekstas, int plotis, std::string &out)
Prideda tekstą prie išvesties eilutės, sulygiuodama jį nurodytu pločiu.

8.6.1 Funkcijos Dokumentacija

8.6.1.1 failoGeneravimas()

```

void failoGeneravimas (
    int studentuKiekis,
    int ndKiekis)

```

Generuoja failą su studentų duomenimis.

Parametrai

<i>studentuKiekis</i>	Kiek studentų generuoti.
<i>ndKiekis</i>	Kiek namų darbų rezultatų generuoti kiekvienam studentui.

Apibrėžimas failo [apdorojimas.cpp](#) eilutėje 3.

8.6.1.2 setw()

```
void setw (  
    const std::string & tekstas,  
    int plotis,  
    std::string & out)
```

Prideda tekstą prie išvesties eilutės, sulygiuodama jį nurodytu pločiu.

Parametrai

<i>tekstas</i>	Tekstas
----------------	---------

<i>plotis</i>	Minimalus eilutės plotis.
<i>out</i>	Nuoroda į išvesties eilutę.

Apibrėžimas failo [apdorojimas.cpp](#) eilutėje 29.

8.7 apdorojimas.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef APDOROJIMAS_H
00002 #define APDOROJIMAS_H
00003
00004 #include <algorithm>
00005 #include <format>
00006 #include "studentas.h"
00007 #include "random.h"
00008 #include <fstream>
00009 #include <list>
00010
00016 void failoGeneravimas(int studentuKiekis, int ndKiekis);
00017
00024 void setw(const std::string &tekstas, int plotis, std::string& out);
00025
00026 #endif
```

8.8 vector/apdorojimas.hpp Failo Nuoroda

```
#include "apdorojimas.h"
```

Funkcijos

- `template<class T, class Konteineris>`
`void rusiavimasPagal` (Konteineris &studentai, T lambdaFunkcija, bool didejanciai=true)
Rūšiuoja konteinerio studentus pagal nurodytą lambda funkciją.
- `template<class Konteineris>`
`void rusiavimasSkirstymas` (Konteineris &studentai, int rPasirinkimas, bool medianos, int ndKiekis=0)
Rūšiuoja studentus pagal vartotojo pasirinkimą (vardą, pavardę, galutinį pažymį).
- `template<class Konteineris>`
`void skirstymas` (Konteineris &studentai, Konteineris &galvociai, Konteineris &vargsiukai, bool medianos, int ndKiekis=0)
Skirsto studentus į "galvočius" (≥ 5) ir "vargšiukus" (< 5) naudojant judinimą (move).
`template<class Konteineris>`
`void skirstymasStrat1` (Konteineris &studentai, Konteineris &galvociai, Konteineris &vargsiukai, bool medianos, int ndKiekis=0)
1 strategija: skirstymas be perkėlimo (kopijavimas).
- `template<class Konteineris>`
`void skirstymasStrat2` (Konteineris &studentai, Konteineris &vargsiukai, bool medianos, int ndKiekis=0)
2 strategija: naudoja `std::lower_bound`, kai konteineris jau surūšiuotas.
- `template<class Konteineris>`
`void skirstymasStrat3` (Konteineris &studentai, Konteineris &vargsiukai, bool medianos, int ndKiekis=0)
3 strategija: naudoja `std::partition`.

8.8.1 Funkcijos Dokumentacija

8.8.1.1 rusiavimasPagal()

```
template<class T, class Konteineris>
void rusiavimasPagal (
    Konteineris & studentai,
    T lambdaFunkcija,
    bool didejanciai = true)
```

Rūšiuoja konteinerio studentus pagal nurodytą lambda funkciją.

Template Parameters

<i>T</i>	lambda funkcijos tipas.
<i>Konteineris</i>	(list, vector, deque).

Parametrai

<i>studentai</i>	Studentų konteineris.
<i>lambdaFunkcija</i>	Funkcija, grąžinanti rūšiavimo būdą.
<i>didejanciai</i>	Jei true – rūšiuoti didėjančiai, kitu atveju – mažėjančiai.

Apibrėžimas failo [apdorojimas.tpp](#) eilutėje 12.

8.8.1.2 rusiavimasSkirstymas()

```
template<class Konteineris>
void rusiavimasSkirstymas (
    Konteineris & studentai,
    int rPasirinkimas,
    bool medianos,
    int ndKiekis = 0)
```

Rūšiuoja studentus pagal vartotojo pasirinkimą (vardą, pavardę, galutinį pažymį).

Template Parameters

<i>Konteineris</i>	(list, vector, deque).
--------------------	------------------------

Parametrai

<i>studentai</i>	Studentų konteineris.
<i>rPasirinkimas</i>	Rūšiavimo būdas (1-6).
<i>medianos</i>	Ar naudoti medianą galutiniam pažymiui.
<i>ndKiekis</i>	Namų darbų kiekis (jei žinomas).

Apibrėžimas failo [apdorojimas.tpp](#) eilutėje 43.

8.8.1.3 skirstymas()

```
template<class Konteineris>
void skirstymas (
    Konteineris & studentai,
    Konteineris & galvociai,
    Konteineris & vargsiukai,
    bool medianos,
    int ndKiekis = 0)
```

Skirsto studentus į "galvočius" (≥ 5) ir "vargšiukus" (< 5) naudojant judinimą (move).

Template Parameters

<i>Konteineris</i>	(list, vector, deque).
--------------------	------------------------

Parametrai

<i>studentai</i>	Studentų konteineris (ištuštinamas).
<i>galvociai</i>	Konteineris studentams, kurių galutinis ≥ 5 .
<i>vargsiukai</i>	Konteineris studentams, kurių galutinis < 5 .
<i>medianos</i>	Ar naudoti medianą galutiniam pažymiui.
<i>ndKiekis</i>	Namų darbų skaičius.

Apibrėžimas failo [apdorojimas.tpp](#) eilutėje 68.

8.8.1.4 skirstymasStrat1()

```
template<class Konteineris>
void skirstymasStrat1 (
    Konteineris & studentai,
    Konteineris & galvociai,
    Konteineris & vargsiukai,
    bool medianos,
    int ndKiekis = 0)
```

1 strategija: skirstymas be perkėlimo (kopijavimas).

Template Parameters

<i>Konteineris</i>	(list, vector, deque).
--------------------	------------------------

Parametrai

<i>studentai</i>	Studentų konteineris (ištuštinamas).
<i>galvociai</i>	Konteineris studentams, kurių galutinis ≥ 5 .
<i>vargsiukai</i>	Konteineris studentams, kurių galutinis < 5 .

<i>medianos</i>	Ar naudoti medianą galutiniam pažymiui.
<i>ndKiekis</i>	Namų darbų skaičius.

Apibrėžimas failo [apdorojimas.tpp](#) eilutėje 95.

8.8.1.5 skirstymasStrat2()

```
template<class Konteineris>
void skirstymasStrat2 (
    Konteineris & studentai,
    Konteineris & vargsiukai,
    bool medianos,
    int ndKiekis = 0)
```

2 strategija: naudoja `std::lower_bound`, kai konteineris jau surūšiuotas.

Template Parameters

<i>Konteineris</i>	(list, vector, deque).
--------------------	------------------------

Parametrai

<i>studentai</i>	Surūšiuotas studentų konteineris (išsaugomi "galvočiai", kurių galutinis ≥ 5).
<i>vargsiukai</i>	Konteineris studentams, kurių galutinis < 5 .
<i>medianos</i>	Ar naudoti medianą galutiniam pažymiui.
<i>ndKiekis</i>	Namų darbų skaičius.

Apibrėžimas failo [apdorojimas.tpp](#) eilutėje 121.

8.8.1.6 skirstymasStrat3()

```
template<class Konteineris>
void skirstymasStrat3 (
    Konteineris & studentai,
    Konteineris & vargsiukai,
    bool medianos,
    int ndKiekis = 0)
```

3 strategija: naudoja `std::partition`.

Template Parameters

<i>Konteineris</i>	(list, vector, deque).
--------------------	------------------------

Parametrai

<i>studentai</i>	Surūšiuotas studentų konteineris (išsaugomi "galvočiai", kurių galutinis ≥ 5).
------------------	--

<i>vargsiukai</i>	Konteineris studentams, kurių galutinis <5.
<i>medianos</i>	Ar naudoti medianą galutiniam pažymiui.
<i>ndKiekis</i>	Namų darbų skaičius.

Apibrėžimas failo `apdorojimas.hpp` eilutėje 140.

8.9 apdorojimas.hpp

Eiti į šio failo dokumentaciją.

```

00001 #include "apdorojimas.h"
00002
00011 template<class T, class Konteineris>
00012 void rusiavimasPagal(Konteineris &studentai, T lambdaFunkcija, bool didejanciai = true)
00013 {
00014     if constexpr(std::is_same_v<Konteineris, std::list<Studentas>>){
00015         if(didejanciai){
00016             studentai.sort([lambdaFunkcija](const Studentas &A, const Studentas &B){
00017                 return lambdaFunkcija(A) < lambdaFunkcija(B);
00018             });
00019         } else {
00020             studentai.sort([lambdaFunkcija](const Studentas &A, const Studentas &B){
00021                 return lambdaFunkcija(A) > lambdaFunkcija(B);
00022             });
00023         }
00024     } else {
00025         std::sort(studentai.begin(), studentai.end(),
00026             [didejanciai, lambdaFunkcija](const Studentas &A, const Studentas &B)
00027             {
00028                 return didejanciai ? lambdaFunkcija(A) < lambdaFunkcija(B) : lambdaFunkcija(A) >
00029                     lambdaFunkcija(B);
00030             });
00031     }
00032 }
00033
00042 template<class Konteineris>
00043 void rusiavimasSkirstymas(Konteineris &studentai, int rPasirinkimas, bool medianos, int ndKiekis = 0){
00044     switch(rPasirinkimas){
00045         case 1: rusiavimasPagal(studentai, [](const Studentas &studentas){return
00046             studentas.getVardas();}); break;
00047         case 2: rusiavimasPagal(studentai, [](const Studentas &studentas){return
00048             studentas.getVardas();}, false); break;
00049         case 3: rusiavimasPagal(studentai, [](const Studentas &studentas){return
00050             studentas.getPavarde();}); break;
00051         case 4: rusiavimasPagal(studentai, [](const Studentas &studentas){return
00052             studentas.getPavarde();}, false); break;
00053         case 5: rusiavimasPagal(studentai, [medianos, ndKiekis](const Studentas &studentas){return
00054             studentas.galutinis(medianos, ndKiekis);}); break;
00055         case 6: rusiavimasPagal(studentai, [medianos, ndKiekis](const Studentas &studentas){return
00056             studentas.galutinis(medianos, ndKiekis);}, false); break;
00057         default:
00058         {
00059             break;
00060         }
00061     }
00062 }
00063
00067 template<class Konteineris>
00068 void skirstymas(Konteineris &studentai, Konteineris &galvociai, Konteineris &vargsiukai, bool
00069     medianos, int ndKiekis = 0)
00070 {
00071     for(Studentas &A : studentai)
00072     {
00073         if(A.galutinis(medianos, ndKiekis)<5){
00074             vargsiukai.push_back(std::move(A));
00075         } else {
00076             galvociai.push_back(std::move(A));
00077         }
00078     }
00079     studentai.clear();
00080     if constexpr(!std::is_same_v<Konteineris, std::list<Studentas>>){
00081         vargsiukai.shrink_to_fit();
00082         galvociai.shrink_to_fit();
00083     }
00084 }

```

```

00084
00094 template<class Konteineris>
00095 void skirstymasStrat1(Konteineris &studentai, Konteineris &galvociai, Konteineris &vargsiukai, bool
medianos, int ndKiekis = 0)
00096 {
00097     for(Studentas &A : studentai)
00098     {
00099         if(A.galutinis(medianos, ndKiekis) < 5) {
00100             vargsiukai.push_back(A);
00101         } else {
00102             galvociai.push_back(A);
00103         }
00104     }
00105     studentai.clear();
00106     if constexpr(!std::is_same_v<Konteineris, std::list<Studentas> >){
00107         vargsiukai.shrink_to_fit();
00108         galvociai.shrink_to_fit();
00109     }
00110 }
00111
00120 template<class Konteineris>
00121 void skirstymasStrat2(Konteineris &studentai, Konteineris &vargsiukai, bool medianos, int ndKiekis =
0){
00122     auto it = std::lower_bound(studentai.begin(), studentai.end(), 5, [medianos, ndKiekis](const
Studentas &A, int skaic)
00123     {
00124         return A.galutinis(medianos, ndKiekis) < skaic;
00125     }
00126     );
00127     vargsiukai.insert(vargsiukai.end(), std::make_move_iterator(studentai.begin()),
std::make_move_iterator(it));
00128     studentai.erase(studentai.begin(), it);
00129 }
00130
00139 template<class Konteineris>
00140 void skirstymasStrat3(Konteineris &studentai, Konteineris &vargsiukai, bool medianos, int ndKiekis =
0){
00141     auto it = std::partition(studentai.begin(), studentai.end(),
00142     [medianos, ndKiekis](const Studentas& A){
00143         return A.galutinis(medianos, ndKiekis) >= 5;
00144     }
00145     );
00146     vargsiukai.insert(vargsiukai.end(), std::make_move_iterator(it),
std::make_move_iterator(studentai.end()));
00147     studentai.erase(it, studentai.end());
00148 }

```

8.10 vector/isvestis.cpp Failo Nuoroda

```

#include "isvestis.h"
#include "ivestis.h"

```

Funkcijos

- int **menu** ()
Parodo pagrindinį meniu ir grąžina vartotojo pasirinkimą.
- int **rusiavimoPasirinkimas** ()
Vartotojo pasirinkimas, pagal ką rūšiuoti studentus.
- int **gautiTipaPasirinkima** ()
Pasirinkimas, kokį konteinerį naudoti testavimui.
- int **testavimoPasirinkimas** ()
Kiek kartų kartoti testavimą.
- int **strategijosPasirinkimas** ()
Pasirinkimas, kurią skirstymo strategiją naudoti.
- int **studentuPasirinkimas** ()
Įvedamas norimas studentų skaičius generavimui.
- int **ndPasirinkimas** ()

- Įvedamas namų darbų rezultatų skaičius generavimui.*
- bool [medianosUzklausa](#) ()
Ar skaičiuoti galutinį pažymį naudojant medianą?
- bool [failoUzklausa](#) ()
Ar išvesti rezultatus į failą?
- int [lietuviskosRaides](#) (const std::string &eilute)
Suskaičiuoja, kiek papildomų baitų užima lietuviškos raidės (UTF-8).
- void [failoPasirinkimas](#) (int &rezervas, bool &egzistuoja, std::string &failoPavadinimas, const std::string &vieta)
Leidžia vartotojui pasirinkti failą iš esamų .txt failų programos direktorijoje.

8.10.1 Funkcijos Dokumentacija

8.10.1.1 failoPasirinkimas()

```
void failoPasirinkimas (
    int & rezervas,
    bool & egzistuoja,
    std::string & failoPavadinimas,
    const std::string & vieta = ".")
```

Leidžia vartotojui pasirinkti failą iš esamų .txt failų programos direktorijoje.

Parametrai

<i>rezervas</i>	Išvedamas rezervavimo dydis (jei failo pavadinime yra skaičius).
<i>egzistuoja</i>	Ar rastas bent vienas tinkamas failas.
<i>failoPavadinimas</i>	Pasirinkto failo pavadinimas.
<i>vieta</i>	direktorija, kurioje numatyta ieškoti failų (numatyta ".").

Apibrėžimas failo [isvestis.cpp](#) eilutėje 65.

8.10.1.2 failoUzklausa()

```
bool failoUzklausa ()
```

Ar išvesti rezultatus į failą?

Gražina

true – į failą, false – į terminalą.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 44.

8.10.1.3 gautiTipoPasirinkima()

```
int gautiTipoPasirinkima ()
```

Pasirinkimas, kokį konteinerį naudoti testavimui.

Gražina

1 – vector, 2 – deque, 3 – list.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 14.

8.10.1.4 lietuiskosRaides()

```
int lietuiskosRaides (  
    const std::string & eilute)
```

Suskaičiuoja, kiek papildomų baitų užima lietuviškos raidės (UTF-8).

Parametrai

<i>eilutė</i>	- tekstas.
---------------	------------

Gražina

Papildomų baitų skaičius (eilutės ilgis – simbolių skaičius).

Apibrėžimas failo [isvestis.cpp](#) eilutėje 50.

8.10.1.5 medianosUzklausa()

```
bool medianosUzklausa ()
```

Ar skaičiuoti galutinį pažymį naudojant medianą?

Gražina

true – medianą, false – vidurkį.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 39.

8.10.1.6 menu()

```
int menu ()
```

Parodo pagrindinį meniu ir gražina vartotojo pasirinkimą.

Gražina

Pasirinkimo numeris (1-8).

Apibrėžimas failo [isvestis.cpp](#) eilutėje 4.

8.10.1.7 ndPasirinkimas()

```
int ndPasirinkimas ()
```

Įvedamas namų darbų rezultatų skaičius generavimui.

Gražina

nd rezultatų skaičius.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 34.

8.10.1.8 rusiavimoPasirinkimas()

```
int rusiavimoPasirinkimas ()
```

Vartotojo pasirinkimas, pagal ką rūšiuoti studentus.

Gražina

Pasirinkimas (1-6).

Apibrėžimas failo [isvestis.cpp](#) eilutėje 9.

8.10.1.9 strategijosPasirinkimas()

```
int strategijosPasirinkimas ()
```

Pasirinkimas, kurią skirstymo strategiją naudoti.

Gražina

0-3.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 24.

8.10.1.10 studentuPasirinkimas()

```
int studentuPasirinkimas ()
```

Įvedamas norimas studentų skaičius generavimui.

Gražina

Studentų skaičius.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 29.

8.10.1.11 testavimoPasirinkimas()

```
int testavimoPasirinkimas ()
```

Kiek kartų kartoti testavimą.

Gražina

Kartų skaičius.

Apibrėžimas failo `isvestis.cpp` eilutėje 19.

8.11 isvestis.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "isvestis.h"
00002 #include "ivestis.h"
00003
00004 int menu()
00005 {
00006     return gautiSkaiciu("Pasirinkite programos eiga:\n1 - ranka,\n2 - generuoti tik pazymius,\n3 -
    generuoti studentu vardus, pavardes ir pazymius,\n4 - skaityti studentus is failo,\n5 - testavimas su
    failais,\n6 - generuoti faila,\n7 - testuoti klase Studentas,\n8 - baigti darba: ", 1, 8);
00007 }
00008
00009 int rusiavimoPasirinkimas()
00010 {
00011     return gautiSkaiciu("Pasirinkite pagal ka rusiuoti: \n1 - varda (A->Z),\n2 - varda (Z->A),\n3 -
    pavarde (A->Z),\n4 - pavarde (Z->A),\n5 - galutini pazymi didėjančiai (1->10),\n6 - galutini pazymi
    mažėjančiai (10->1)\n", 1, 6);
00012 }
00013
00014 int gautiTipoPasirinkima()
00015 {
00016     return gautiSkaiciu("Koki konteinerio tipa norite testuoti?\n1 - vektorių,\n2 - deque,\n3 - list:
    ", 1, 3);
00017 }
00018
00019 int testavimoPasirinkimas()
00020 {
00021     return gautiSkaiciu("Kiek kartų testuoti failą? (1-100):", 1, 100);
00022 }
00023
00024 int strategijosPasirinkimas()
00025 {
00026     return gautiSkaiciu("Pagal kokia strategija testuoti?\n0 - pradiniame relize naudota
    strategija,\n1 - pirma strategija,\n2 - antra strategija,\n3 - trečia strategija:", 0, 3);
00027 }
00028
00029 int studentuPasirinkimas()
00030 {
00031     return gautiSkaiciu("Iveskite norima generuoti studentu kieki (1-10000000):", 1, 10000000);
00032 }
00033
00034 int ndPasirinkimas()
00035 {
00036     return gautiSkaiciu("Iveskite norima generuoti namu darbu rezultatu kieki (0-10000000): ", 0,
    10000000);
00037 }
00038
00039 bool medianosUzklausa()
00040 {
00041     return gautiPatvirtinima("Skaiciuoti tik medianas? Jei ne, tai galutinis rezultatas bus
    skaičiuojamas su vidurkiu");
00042 }
00043
00044 bool failoUzklausa()
00045 {
00046     return gautiPatvirtinima("Ar išvesti į terminalą? Jei ne, tai bus išvedama į rezultatai.txt
    failą");
00047 }
00048
```



```

00049 // apskaiciuoti kiek string su lietuviskais raidėmis sudaro baitų, kadangi viena lietuviska raidė - 2
      baitai, o ne 1 baitas. Kitaip sakant vardas Ažuolas turi 7 raides, o jį sudaro 8 baitai, o setw mato
      baitus, tai jei setw(20), tai jis pridės 12 tusčių tarpų, o ne 13.
00050 int lietuviskosRaides(const std::string& eilute)
00051 {
00052     int simboliuKiekis = 0;
00053     for (char c : eilute)
00054     {
00055         // jei baitas neprasideda su 10xxxxxx, tai naujas simbolis
00056         if ((c & 0xC0) != 0x80)
00057         { // paprastas ASCII simbolis prasideda su 0, keliu baitu pvz lietuviskos raides prasideda su
11 arba 111 arba 1111, priklausomai nuo kodavimo | 0xC0 = 11000000, 0x80 = 10000000. & (AND) bit'u
      operacija atranda ar c prasideda su 0 ar 1. antras, trečias ar ketvirtas baitas UTF-8 kodavime visad
      prasides su 10xxxxxx
00058             simboliuKiekis++;
00059         }
00060     }
00061     // grazinam trukstama isvesties ploti.
00062     return eilute.length() - simboliuKiekis;
00063 }
00064
00065 void failoPasirinkimas(int &rezervas, bool &egzistuoja, std::string &failoPavadinimas, const
      std::string& vieta)
00066 {
00067     std::vector<std::filesystem::directory_entry> failai;
00068     std::vector<int> rezervai;
00069
00070     for(auto &failas : std::filesystem::directory_iterator(vieta))
00071     {
00072         if(!failas.is_regular_file())
00073             continue;
00074
00075         if(failas.path().extension() != ".txt")
00076             continue;
00077
00078         std::string vardas = failas.path().filename().string();
00079
00080         if(vardas == "rezultatai.txt" || vardas == "galvociiai.txt" || vardas == "vargsiukai.txt")
00081             continue;
00082
00083         if(vardas.find("studentai", 0) == 0)
00084         {
00085             std::string skaicius = vardas.substr(9, vardas.size() - 9 - 4);
00086
00087             if(skaicius.empty() || !std::all_of(skaicius.begin(), skaicius.end(), [](unsigned char
      simbolis)
00088                 {
00089                     return std::isdigit(simbolis);
00090                 })))
00091             {
00092                 rezervai.push_back(0);
00093             }
00094             else
00095             {
00096                 rezervai.push_back(std::stoi(skaicius));
00097             }
00098         } else {
00099             rezervai.push_back(0);
00100         }
00101
00102         failai.push_back(failas);
00103     }
00104
00105     if(failai.empty())
00106     {
00107         std::cout << "Nerasta tekstinių failų vietoje." << vieta << "\n";
00108         egzistuoja = false;
00109     } else {
00110
00111         std::cout << "Pasirinkite failą:\n";
00112         for(int i = 0; i < failai.size(); i++)
00113         {
00114             std::cout << (i + 1) << ": " << failai.at(i).path().filename().string() << "\n";
00115         }
00116
00117         int pasirinkimas = gautiSkaiciu("Įveskite failo numerį: ", 1, failai.size(), false);
00118         failoPavadinimas = failai.at(pasirinkimas-1).path().filename().string();
00119         rezervas = rezervai.at(pasirinkimas-1);
00120     }
00121 }

```

8.12 vector/isvestis.h Failo Nuoroda

```
#include "studentas.h"
#include "apdorojimas.h"
#include <iomanip>
#include <iostream>
#include <fstream>
#include <filesystem>
#include <format>
```

Funkcijos

- int [menu](#) ()
Parodo pagrindinį meniu ir grąžina vartotojo pasirinkimą.
- int [rusiavimoPasirinkimas](#) ()
Vartotojo pasirinkimas, pagal ką rūšiuoti studentus.
- int [gautiTipaPasirinkima](#) ()
Pasirinkimas, kokį konteinerį naudoti testavimui.
- int [testavimoPasirinkimas](#) ()
Kiek kartų kartoti testavimą.
- int [strategijosPasirinkimas](#) ()
Pasirinkimas, kurią skirstymo strategiją naudoti.
- int [studentuPasirinkimas](#) ()
Įvedamas norimas studentų skaičius generavimui.
- int [ndPasirinkimas](#) ()
Įvedamas namų darbų rezultatų skaičius generavimui.
- bool [medianosUzklausa](#) ()
Ar skaičiuoti galutinį pažymį naudojant medianą?
- bool [failoUzklausa](#) ()
Ar išvesti rezultatus į failą?
- int [lietuviskosRaides](#) (const std::string &eilute)
Suskaičiuoja, kiek papildomų baitų užima lietuviškos raidės (UTF-8).
- void [failoPasirinkimas](#) (int &rezervas, bool &egzistuoja, std::string &failoPavadinimas, const std::string &vieta=".")
Leidžia vartotojui pasirinkti failą iš esamų .txt failų programos direktorijoje.

8.12.1 Funkcijos Dokumentacija

8.12.1.1 failoPasirinkimas()

```
void failoPasirinkimas (
    int & rezervas,
    bool & egzistuoja,
    std::string & failoPavadinimas,
    const std::string & vieta = ".")
```

Leidžia vartotojui pasirinkti failą iš esamų .txt failų programos direktorijoje.

Parametrai

<i>rezervas</i>	Išvedamas rezervavimo dydis (jei failo pavadinime yra skaičius).
-----------------	--

<i>egzistuoja</i>	Ar rastas bent vienas tinkamas failas.
<i>failoPavadinimas</i>	Pasirinkto failo pavadinimas.
<i>vieta</i>	direktorija, kurioje numatyta ieškoti failų (numatyta ".").

Apibrėžimas failo [isvestis.cpp](#) eilutėje 65.

8.12.1.2 failoUzklausa()

```
bool failoUzklausa ()
```

Ar išvesti rezultatus į failą?

Gražina

true – į failą, false – į terminalą.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 44.

8.12.1.3 gautiTipoSirinkima()

```
int gautiTipoSirinkima ()
```

Pasirinkimas, kokį konteinerį naudoti testavimui.

Gražina

1 – vector, 2 – deque, 3 – list.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 14.

8.12.1.4 lietuiskosRaides()

```
int lietuiskosRaides (
    const std::string & eilute)
```

Suskaičiuoja, kiek papildomų baitų užima lietuviškos raidės (UTF-8).

Parametrai

<i>eilutė</i>	- tekstas.
---------------	------------

Gražina

Papildomų baitų skaičius (eilutės ilgis – simbolių skaičius).

Apibrėžimas failo [isvestis.cpp](#) eilutėje 50.

8.12.1.5 medianosUzklausa()

```
bool medianosUzklausa ()
```

Ar skaičiuoti galutinį pažymį naudojant medianą?

Gražina

true – medianą, false – vidurkį.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 39.

8.12.1.6 menu()

```
int menu ()
```

Parodo pagrindinį meniu ir gražina vartotojo pasirinkimą.

Gražina

Pasirinkimo numeris (1-8).

Apibrėžimas failo [isvestis.cpp](#) eilutėje 4.

8.12.1.7 ndPasirinkimas()

```
int ndPasirinkimas ()
```

Įvedamas namų darbų rezultatų skaičius generavimui.

Gražina

nd rezultatų skaičius.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 34.

8.12.1.8 rusiavimoPasirinkimas()

```
int rusiavimoPasirinkimas ()
```

Vartotojo pasirinkimas, pagal ką rūšiuoti studentus.

Gražina

Pasirinkimas (1-6).

Apibrėžimas failo [isvestis.cpp](#) eilutėje 9.

8.12.1.9 strategijosPasirinkimas()

```
int strategijosPasirinkimas ()
```

Pasirinkimas, kurią skirstymo strategiją naudoti.

Gražina

0-3.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 24.

8.12.1.10 studentuPasirinkimas()

```
int studentuPasirinkimas ()
```

Įvedamas norimas studentų skaičius generavimui.

Gražina

Studentų skaičius.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 29.

8.12.1.11 testavimoPasirinkimas()

```
int testavimoPasirinkimas ()
```

Kiek kartų kartoti testavimą.

Gražina

Kartų skaičius.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 19.

8.13 isvestis.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef ISVESTIS_H
00002 #define ISVESTIS_H
00003
00004 #include "studentas.h"
00005 #include "apdorojimas.h"
00006 #include <iomanip>
00007 #include <iostream>
00008 #include <fstream>
00009 #include <filesystem>
00010 #include <format>
00011
00016 int menu();
00017
00022 int rusiavimoPasirinkimas();
00023
00028 int gautiTipoPasirinkima();
00029
00034 int testavimoPasirinkimas();
00035
00040 int strategijosPasirinkimas();
00041
00046 int studentuPasirinkimas();
00047
00052 int ndPasirinkimas();
00053
00058 bool medianosUzklausa();
00059
00064 bool failoUzklausa();
00065
00071 int lietuviskosRaides(const std::string& eilute);
00072
00080 void failoPasirinkimas(int &rezervas, bool &egzistuoja, std::string &failoPavadinimas, const
    std::string& vieta = ".");
00081
00082 #endif
```

8.14 vector/isvestis.hpp Failo Nuoroda

```
#include "isvestis.h"
```

Funkcijos

- `template<class Konteineris>`
`void isvestis (Konteineris &A, bool medianos, bool failas, const std::string &failoPavadinimas="rezultatai.txt",`
`int ndKiekis=0)`
Išveda studentų sąrašą į failą arba ekraną.

8.14.1 Funkcijos Dokumentacija

8.14.1.1 isvestis()

```
template<class Konteineris>
void isvestis (
    Konteineris & A,
    bool medianos,
    bool failas,
```

```
const std::string & failoPavadinimas = "rezultatai.txt",  
int ndKiekis = 0)
```

Išveda studentų sąrašą į failą arba ekraną.

Template Parameters

<i>Konteineris</i>	(list, vector, deque).
--------------------	------------------------

Parametrai

<i>A</i>	Studentų konteineris.
----------	-----------------------

<i>medianos</i>	Ar naudoti medianą galutiniam pažymiui.
<i>failas</i>	Jei true – išvesti į failą "rezultatai.txt", kitu atveju – į std::cout.
<i>failoPavadinimas</i>	Išvesties failo pavadinimas (numatyta "rezultatai.txt").
<i>ndKiekis</i>	Namų darbų skaičius.

Apibrėžimas failo [isvestis.hpp](#) eilutėje 13.

8.15 isvestis.hpp

Eiti į šio failo dokumentaciją.

```

00001 #include "isvestis.h"
00002
00012 template<class Konteineris>
00013 void isvestis(Konteineris &A, bool medianos, bool failas, const std::string &failoPavadinimas =
    "rezultatai.txt", int ndKiekis = 0)
00014 {
00015     std::string out;
00016
00017     setw("Vardas", 20, out);
00018     setw("Pavardė", 21, out);
00019     out += "Galutinis (Vid.) / Galutinis (Med.)\n";
00020     out += std::string(75, '-') + "\n";
00021     for(const Studentas &X : A)
00022     {
00023         int vardoPlotis = 20 + lietuviskosRaides(X.getVardas());
00024         int pavardesPlotis = 20 + lietuviskosRaides(X.getPavarde());
00025
00026         setw(X.getVardas(), vardoPlotis, out);
00027         setw(X.getPavarde(), pavardesPlotis, out);
00028         if (medianos)
00029         {
00030             setw("x.xx", 19, out);
00031             out += std::format("{:.2f}", X.galutinis(medianos, ndKiekis));
00032             out += '\n';
00033         }
00034         else
00035         {
00036             setw(std::format("{:.2f}", X.galutinis(medianos, ndKiekis)), 19, out);
00037             out += "y.yy\n";
00038         }
00039     }
00040     if(failas){
00041         std::ofstream fout(failoPavadinimas);
00042         fout << out;
00043         fout.close();
00044     } else {
00045         std::cout << out;
00046     }
00047 }
```

8.16 vector<Ivestis>.cpp Failo Nuoroda

```
#include "investis.h"
```

Funkcijos

- bool [gautiPatvirtinima](#) (const std::string &pranesimas)
Paprašo vartotojo patvirtinimo (y/n).
- std::vector< [Studentas](#) > [investiStudentus](#) ()
Nuskaito studentus iš standartinės įvesties iki EOF.

- int [gautiSkaiciu](#) (const std::string &pranešimas, int min, int max, bool galiButiTuscia)
Nuskaityto sveikąjį skaičių iš konsolės su patikrinimais.
- bool [arTikSkaicius](#) (const std::string &eilute)
Patikrina, ar eilutę sudaro tik skaitmenys ir tarpai.
- void [cinEOFgaudymas](#) ()
Apdoroja EOF atvejį cin sraute.

8.16.1 Funkcijos Dokumentacija

8.16.1.1 arTikSkaicius()

```
bool arTikSkaicius (
    const std::string & s)
```

Patikrina, ar eilutę sudaro tik skaitmenys ir tarpai.

Parametrai

s	Tikrinama eilutė.
---	-------------------

Gražina

true jei taip.

Apibrėžimas failo [ivestis.cpp](#) eilutėje 76.

8.16.1.2 cinEOFgaudymas()

```
void cinEOFgaudymas ()
```

Apdoroja EOF atvejį cin sraute.

Apibrėžimas failo [ivestis.cpp](#) eilutėje 85.

8.16.1.3 gautiPatvirtinima()

```
bool gautiPatvirtinima (
    const std::string & pranesimas)
```

Paprašo vartotojo patvirtinimo (y/n).

Parametrai

pranesimas	Tekstas, rodomas prieš įvedimą.
------------	---------------------------------

Gražina

true jei 'y', false jei 'n'.

Apibrėžimas failo [ivestis.cpp](#) eilutėje 3.

8.16.1.4 gautiSkaiciu()

```
int gautiSkaiciu (
    const std::string & pranešimas,
    int min,
    int max,
    bool galiButiTuscia = false)
```

Nuskaito sveikąjį skaičių iš konsolės su patikrinimais.

Parametrai

<i>pranešimas</i>	- tekstas, rodomas prieš įvedimą.
<i>min</i>	Mažiausia leistina reikšmė.
<i>max</i>	Didžiausia leistina reikšmė.
<i>galiButiTuscia</i>	Ar galima įvesti tuščią eilutę (grąžina -1).

Gražina

Įvestas skaičius.

Apibrėžimas failo [investis.cpp](#) eilutėje 43.

8.16.1.5 ivestiStudentus()

```
std::vector< Studentas > ivestiStudentus ()
```

Nuskaito studentus iš standartinės įvesties iki EOF.

Gražina

Vektorius su nuskaitytais studentais.

Apibrėžimas failo [investis.cpp](#) eilutėje 29.

8.17 investis.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "investis.h"
00002
00003 bool gautiPatvirtinima(const std::string &pranesimas)
00004 {
00005     std::string ivestis;
00006     while (true) {
00007         std::cout << pranesimas << " (y/n): ";
00008
00009         if (!std::getline(std::cin, ivestis)) {
00010             cinEOFgaudymas();
00011             continue;
00012         }
00013
00014         ivestis.erase(0, ivestis.find_first_not_of(" \t")); // randa pirma simboli kuris nera tarpas
00015         arba tabuliacija, tada trina nuo 0-inio indekso iki rasto simbolio.
```

```

00015         igestis.erase(igestis.find_last_not_of(" \t") + 1); // randa pirma simboli kuris nera tarpas
        arba tabuliacija nuo galo ir istrina viska po to
00016
00017         // igesties ilgio patikrinimas ir konvertavimas
00018         if (igestis.length() == 1) {
00019             // igesti vercia i mazaja | to lower tikisi unsigned char arba EOF pagal standarta,
            static_cast keicia char interpretavima i unsigned char.
00020             if (std::tolower(static_cast<unsigned char>(igestis[0])) == 'y') return true;
00021             if (std::tolower(static_cast<unsigned char>(igestis[0])) == 'n') return false;
00022         }
00023
00024         // jei igestis neteisinga, t.y. nieko nebuvo returninta, tai prompt'ina vartotoja vel igesti y
        ar n!
00025         std::cout << "Neteisinga igestis! Prašome igesti tik 'y' arba 'n'.\\n";
00026     }
00027 }
00028
00029 std::vector<Studentas> igestiStudentus()
00030 {
00031     std::vector<Studentas> studentai;
00032     while(true)
00033     {
00034         Studentas A;
00035         std::cin >> A;
00036         if(!std::cin) break;
00037         studentai.push_back(std::move(A));
00038     }
00039     std::cin.clear();
00040     return studentai;
00041 }
00042
00043 int gautiSkaiciu(const std::string &pranešimas, int min, int max, bool galiButiTuscia /* = false */)
00044 {
00045     std::string igestis;
00046     while (true) {
00047         std::cout << pranešimas;
00048
00049         if (!std::getline(std::cin, igestis)) {
00050             if (std::cin.eof())
00051                 cinEOFgaudymas();
00052             continue;
00053         }
00054
00055         // igesties nutraukimas su ENTER
00056         if (galiButiTuscia && igestis.empty()) return -1;
00057
00058         try {
00059             //
00060             if (!arTikSkaicius(igestis)) throw std::invalid_argument("Ne skaičius");
00061
00062             int skaicius = std::stoi(igestis);
00063
00064             //tikriname ar igestas skaicius atitinka nuo maziausio leistino iki didžiausio leistino
00065             if (skaicius >= min && skaicius <= max) {
00066                 return skaicius;
00067             } else {
00068                 std::cout << "Klaida, skaičius turi būti tarp " << min << " ir " << max << "\\n";
00069             }
00070         } catch (...) {
00071             std::cout << "Klaida, įveskite sveikąjį skaičių!\\n";
00072         }
00073     }
00074 }
00075
00076 bool arTikSkaicius(const std::string& eilute)
00077 { // jei eilute tuscia grazinama false, std::all_of pereina nuo eilutes.begin() pradžios iki galo
        eilutes.end() per kiekviena simboli, kiekvienam simboliui jei jis skaicius ar tarpas grazina true, jei
        tai tiesiog raide – grazinama false ir toliau eilute nebetikrinama
00078     return !eilute.empty() &&
00079         std::all_of(eilute.begin(), eilute.end(), [](unsigned char simbolis)
00080         {
00081             return std::isdigit(simbolis) || std::isspace(simbolis);
00082         });
00083 }
00084
00085 void cinEOFgaudymas()
00086 {
00087     if (std::cin.eof())
00088     { // apsauga nuo CTRL+D (linux), CTRL+Z (windows)
00089         throw std::runtime_error("Įvesties pabaiga (EOF). Darbas su programa baigtas");
00090     }
00091     std::cin.clear(); // atstatome std::cin fail flag'a
00092 }

```

8.18 vector/ivestis.h Failo Nuoroda

```
#include "studentas.h"
#include <iostream>
#include <fstream>
#include <sstream>
#include <algorithm>
#include "ivestis.tpp"
```

Funkcijos

- bool [gautiPatvirtinima](#) (const std::string &pranesimas)
Paprašo vartotojo patvirtinimo (y/n).
- std::vector< [Studentas](#) > [ivestiStudentus](#) ()
Nuskaityti studentus iš standartinės įvesties iki EOF.
- bool [skaitymas](#) ([Studentas](#) &A)
Nuskaityti vieną studentą iš srauto (naudojama skaitymui iš failo ar konsolės).
- bool [studentoVardoPavardesIvestis](#) ([Studentas](#) &A, const std::string &eilute)
Nuskaityti vardą ir pavardę iš eilutės.
- void [namuDarbuRezultatulvestis](#) ([Studentas](#) &A)
Nuskaityti namų darbų rezultatus (terminale).
- void [egzaminoRezultatulvestis](#) ([Studentas](#) &A)
Nuskaityti egzamino rezultatą (terminale).
- int [gautiSkaiciu](#) (const std::string &pranešimas, int min, int max, bool galiButiTuscia=false)
Nuskaityti sveikąjį skaičių iš konsolės su patikrinimais.
- bool [arTikSkaicius](#) (const std::string &s)
Patikrina, ar eilutę sudaro tik skaitmenys ir tarpai.
- template<class Konteineris>
Konteineris [skaitymasIsFailo](#) (const std::string &failoPavadinimas, int &ndKiekis, int rezervas)
Nuskaityti studentus iš failo.
- void [cinEOFgaudymas](#) ()
Apdoroja EOF atvejį cin sraute.

8.18.1 Funkcijos Dokumentacija

8.18.1.1 arTikSkaicius()

```
bool arTikSkaicius (
    const std::string & s)
```

Patikrina, ar eilutę sudaro tik skaitmenys ir tarpai.

Parametrai

s	Tikrinama eilutė.
---	-------------------

Gražina

true jei taip.

Apibrėžimas failo [ivestis.cpp](#) eilutėje 76.

8.18.1.2 cinEOFgaudymas()

```
void cinEOFgaudymas ()
```

Apdoroja EOF atvejį cin sraute.

Apibrėžimas failo [ivestis.cpp](#) eilutėje 85.

8.18.1.3 egzaminoRezultatIvestis()

```
void egzaminoRezultatoIvestis (
    Studentas & A)
```

Nuskaityto egzamino rezultatą (terminale).

Parametrai

A	Studento objektas.
---	--------------------

8.18.1.4 gautiPatvirtinima()

```
bool gautiPatvirtinima (
    const std::string & pranesimas)
```

Paprašo vartotojo patvirtinimo (y/n).

Parametrai

<i>pranesimas</i>	Tekstas, rodomas prieš įvedimą.
-------------------	---------------------------------

Gražina

true jei 'y', false jei 'n'.

Apibrėžimas failo [ivestis.cpp](#) eilutėje 3.

8.18.1.5 gautiSkaiciu()

```
int gautiSkaiciu (
    const std::string & pranešimas,
    int min,
    int max,
    bool galiButiTuscia = false)
```

Nuskaityto sveikąjį skaičių iš konsolės su patikrinimais.

Parametrai

<i>pranešimas</i>	- tekstas, rodomas prieš įvedimą.
-------------------	-----------------------------------

<i>min</i>	Mažiausia leistina reikšmė.
<i>max</i>	Didžiausia leistina reikšmė.
<i>galiButiTuscia</i>	Ar galima įvesti tuščią eilutę (grąžina -1).

Gražina

Įvestas skaičius.

Apibrėžimas failo [investis.cpp](#) eilutėje 43.

8.18.1.6 ivestiStudentus()

```
std::vector< Studentas > ivestiStudentus ()
```

Nuskaityto studentus iš standartinės įvesties iki EOF.

Gražina

Vektorius su nuskaitytais studentais.

Apibrėžimas failo [investis.cpp](#) eilutėje 29.

8.18.1.7 namuDarbuRezultatuIvestis()

```
void namuDarbuRezultatuIvestis (
    Studentas & A)
```

Nuskaityto namų darbų rezultatus (terminale).

Parametrai

<i>A</i>	Studento objektas.
----------	--------------------

8.18.1.8 skaitymas()

```
bool skaitymas (
    Studentas & A)
```

Nuskaityto vieną studentą iš srauto (naudojama skaitymui iš failo ar konsolės).

Parametrai

<i>A</i>	Studento objektas, kur bus įrašyti duomenys.
----------	--

Gražina

true jei nuskaitymas sėkmingas.

8.18.1.9 skaitymasIsFailo()

```
template<class Konteineris>
Konteineris skaitymasIsFailo (
    const std::string & failoPavadinimas,
    int & ndKiekis,
    int rezervas)
```

Nuskaito studentus iš failo.

Template Parameters

<i>Konteineris</i>	(vector, deque, list).
--------------------	------------------------

Parametrai

<i>failoPavadinimas</i>	Failo pavadinimas.
<i>ndKiekis</i>	Išvestinis namų darbų skaičius (nustatomas iš antraštės).
<i>rezervas</i>	Preliminarus konteinerio rezervavimo dydis (jei palaikomas).

Gražina

Konteineris su nuskaitytais studentais.

Apibrėžimas failo [ivestis.tpp](#) eilutėje 10.

8.18.1.10 studentoVardoPavardesIvestis()

```
bool studentoVardoPavardesIvestis (
    Studentas & A,
    const std::string & eilute)
```

Nuskaito vardą ir pavardę iš eilutės.

Parametrai

<i>A</i>	Studento objektas.
<i>eilute</i>	Eilutė su vardu ir pavarde.

Gražina

true jei nuskaityta sėkmingai.

8.19 ivestis.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef IVESTIS_H
00002 #define IVESTIS_H
00003
00004 #include "studentas.h"
00005 #include <iostream>
00006 #include <fstream>
00007 #include <sstream>
00008 #include <algorithm>
00009 #include "investis.hpp"
00010
00016 bool gautiPatvirtinima(const std::string &pranesimas);
00017
00022 std::vector<Studentas> ivestiStudentus();
00023
00029 bool skaitymas(Studentas &A);
00030
00037 bool studentoVardoPavardesIvestis(Studentas &A, const std::string &eilute);
00038
00043 void namuDarbuRezultatuIvestis(Studentas &A);
00044
00049 void egzaminoRezultatoIvestis(Studentas &A);
00050
00059 int gautiSkaiciu(const std::string &pranešimas, int min, int max, bool galiButiTuscia = false);
00060
00066 bool arTikSkaicius(const std::string &s);
00067
00076 template<class Konteineris>
00077 Konteineris skaitymasIsFailo(const std::string &failoPavadinimas, int &ndKiekis, int rezervas);
00078
00082 void cinEOFgaudymas();
00083
00084 #endif
```

8.20 vector/ivestis.hpp Failo Nuoroda

Funkcijos

- template<class Konteineris>
Konteineris **skaitymasIsFailo** (const std::string &failoPavadinimas, int &ndKiekis, int rezervas)
Nuskaityto studentus iš failo.

8.20.1 Funkcijos Dokumentacija

8.20.1.1 skaitymasIsFailo()

```
template<class Konteineris>
Konteineris skaitymasIsFailo (
    const std::string & failoPavadinimas,
    int & ndKiekis,
    int rezervas)
```

Nuskaityto studentus iš failo.

Template Parameters

<i>Konteineris</i>	(vector, deque, list).
--------------------	------------------------

Parametrai

<i>failoPavadinimas</i>	Failo pavadinimas.
-------------------------	--------------------

<i>ndKiekis</i>	Išvestinis namų darbų skaičius (nustatomas iš antraštės).
<i>rezervas</i>	Preliminarus konteinerio rezervavimo dydis (jei palaikomas).

Gražina

Konteineris su nuskaitytais studentais.

Apibrėžimas failo `ivestis.hpp` eilutėje 10.

8.21 ivestis.hpp

Eiti į šio failo dokumentaciją.

```

00001
00009 template<class Konteineris>
00010 Konteineris skaitymasIsFailo(const std::string &failoPavadinimas, int &ndKiekis, int rezervas){
00011     Konteineris studentai;
00012     if constexpr(requires(Konteineris konteineris){konteineris.reserve(0);})
00013         studentai.reserve(rezervas);
00014     std::string eil, t;
00015
00016     std::ifstream open_f(failoPavadinimas);
00017     if (!open_f.is_open()) throw std::runtime_error("Klaida: failas \"" + failoPavadinimas + "\"
nerastas.");
00018
00019     if(!std::getline(open_f, eil)) throw std::runtime_error("Klaida: failas tuščias arba
netinkantis");
00020     std::stringstream antraste(eil);
00021     antraste >> t >> t;
00022     while(antraste >> t){
00023         if(t == "Egz." || t == "Egzaminas") break;
00024         ndKiekis++;
00025     }
00026
00027     std::string vardas, pavarde;
00028     int paz;
00029
00030     while (true) {
00031         Studentas studentas;
00032         if(!(studentas.readStudentStream(open_f, ndKiekis))) break;
00033         studentai.push_back(std::move(studentas));
00034     }
00035
00036     open_f.close();
00037     return studentai;
00038 }
```

8.22 vector/main.cpp Failo Nuoroda

```
#include "main.h"
```

Funkcijos

- int `main` ()
- void `failoApdorojimas` (const std::string &failoPavadinimas, int rezervas, int &ndKiekis, bool medianos, int rPasirinkimas, bool failas)

Apdoroja failą: nuskaito, surūšiuoja, išveda.

8.22.1 Funkcijos Dokumentacija

8.22.1.1 failoApdorojimas()

```
void failoApdorojimas (
    const std::string & failoPavadinimas,
    int rezervas,
    int & ndKiekis,
    bool medianos,
    int rPasirinkimas,
    bool failas)
```

Apdoroja failą: nuskaityti, surūšiuoja, išveda.

Parametrai

<i>failoPavadinimas</i>	Failo, iš kurio skaityti, pavadinimas.
<i>rezervas</i>	Rezervavimo dydis konteneriui (jei žinomas).
<i>ndKiekis</i>	Namų darbų skaičius.
<i>medianos</i>	Ar naudoti medianą galutiniam pažymiui.
<i>rPasirinkimas</i>	Rūšiavimo būdas.
<i>failas</i>	Ar išvesti į failą (true) ar į ekraną (false).

Apibrėžimas failo [main.cpp](#) eilutėje 145.

8.22.1.2 main()

```
int main ()
```

Apibrėžimas failo [main.cpp](#) eilutėje 3.

8.23 main.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "main.h"
00002
00003 int main()
00004 {
00005     try
00006     {
00007         #ifdef _WIN32 // Jei kompiliuojama Windows operacinei sistemai nustatyti konsolės įvestį ir
išvestį UTF-8 užkodavimui.
00008         SetConsoleOutputCP(CP_UTF8); // pakeičiame išvesties code page į UTF-8
00009         SetConsoleCP(CP_UTF8); // pakeičiame įvesties code page į UTF-8
00010         #endif
00011         bool veikimas = true;
00012         while(veikimas)
00013         {
00014             //pasirinkima galima tobulint su enumeratorium del type safety ir jei butu norima valdyti
atminti.
00015             int pasirinkimas = menu();
00016             bool failas = false;
00017             bool medianos = false;
00018             if(pasirinkimas<5)
00019             {
00020                 failas = !failoUzklausa();
00021                 medianos = medianosUzklausa();
```

```

00022         } else if(pasirinkimas == 5){
00023             medianos = medianosUzklausa();
00024         }
00025
00026         switch(pasirinkimas){
00027             case 1: // rankinis ivedimas
00028             {
00029                 std::vector<Studentas> studentai = iverstiStudentus();
00030                 skaiciavimas(studentai, medianos); /*
00031                 isvestis(studentai, medianos, failas);
00032                 break;
00033             }
00034             case 2: // tik pazymiu generavimas.
00035             {
00036                 std::vector<Studentas> studentai = iverstiStudentusRandom(pasirinkimas);
00037                 skaiciavimas(studentai, medianos); /*
00038                 isvestis(studentai, medianos, failas);
00039                 break;
00040             }
00041             case 3: // studentu ir pazymiu generavimas;
00042             {
00043                 std::vector<Studentas> studentai = iverstiStudentusRandom(pasirinkimas);
00044                 skaiciavimas(studentai, medianos); /*
00045                 isvestis(studentai, medianos, failas);
00046                 break;
00047             }
00048             case 4: // skaitymas is failo
00049             {
00050                 int ndKiekis = 0;
00051                 int rezervas = 0;
00052                 std::string fPasirinkimas;
00053                 bool egzistuojaFailai = true;
00054                 failoPasirinkimas(rezervas, egzistuojaFailai, fPasirinkimas);
00055                 if(egzistuojaFailai == false) break;
00056                 int rPasirinkimas = rusiavimoPasirinkimas();
00057                 failoApdorojimas(fPasirinkimas, rezervas, ndKiekis, medianos, rPasirinkimas,
00058                 failas);
00059                 break;
00060             }
00061             case 5: // testavimas su failais // kiekvienam test case'e uzkomentuota koda arba jo
00062                 dalis galima atkomentuoti bei keisti parametrus, kad pakeisti kas yra testuojama, kadangi tiksliai
00063                 neapibrezta pagal ka testuoti.
00064             {
00065                 int ndKiekis = 0;
00066                 int rezervas = 0;
00067                 std::string fPasirinkimas;
00068                 bool egzistuojaFailai = true;
00069                 failoPasirinkimas(rezervas, egzistuojaFailai, fPasirinkimas);
00070                 if(egzistuojaFailai == false) break;
00071                 int tipoPasirinkimas = gautiTipoPasirinkima();
00072                 int tPasirinkimas = testavimoPasirinkimas();
00073                 int sPasirinkimas = strategijosPasirinkimas();
00074                 if(tPasirinkimas <= 0) throw std::invalid_argument("Testavimo skaičius turi būti
00075                 daugiau už 0!");
00076                 switch(tipoPasirinkimas){
00077                     case 1:
00078                     {
00079                         failoTestavimas<std::vector<Studentas>>(fPasirinkimas, rezervas,
00080                         tPasirinkimas, sPasirinkimas, ndKiekis, medianos);
00081                         break;
00082                     }
00083                     case 2:
00084                     {
00085                         failoTestavimas<std::deque<Studentas>>(fPasirinkimas, rezervas,
00086                         tPasirinkimas, sPasirinkimas, ndKiekis, medianos);
00087                         break;
00088                     }
00089                     case 3:
00090                     {
00091                         failoTestavimas<std::list<Studentas>>(fPasirinkimas, rezervas,
00092                         tPasirinkimas, sPasirinkimas, ndKiekis, medianos);
00093                         break;
00094                     }
00095                     default:
00096                     {
00097                         }
00098                     }
00099                 }
00100                 break;
00101             }
00102             case 6: // failo generavimas
00103             {
00104                 int studentuKiekis = studentuPasirinkimas();
00105                 int ndKiekis = ndPasirinkimas();
00106                 int tPasirinkimas = testavimoPasirinkimas();
00107                 if(tPasirinkimas <= 0) throw std::invalid_argument("Testavimo skaičius turi būti
00108                 daugiau už 0!");

```

```

00101         double trukme = 0;
00102         for(int i = 0; i < tPasirinkimas+1; i++)
00103         {
00104             if(i!=0)
00105             {
00106                 Timer t;
00107                 failoGeneravimas(studentuKiekis, ndKiekis);
00108                 trukme += t.elapsed();
00109             }
00110             else
00111             {
00112                 failoGeneravimas(studentuKiekis, ndKiekis);
00113             }
00114         }
00115         std::cout << "Failų(-o) generavimas vidutiniškai užtruko: " << trukme/tPasirinkimas
00116         << " s\n";
00117         break;
00118     case 7: // studento klasės testas
00119     {
00120         testas();
00121         break;
00122     }
00123     case 8: // darbo baigtis
00124     {
00125         std::cout << "Darbas su programa baigtas.";
00126         veikimas = false;
00127         return 0;
00128     }
00129     default:
00130     {
00131         std::cout << "How did we get here?" << std::endl; //
00132         https://minecraft.wiki/w/Tutorial:Advancement_guide/How_Did_We_Get_Here%3F
00133         return 0;
00134     }
00135 }
00136 }
00137 catch(const std::exception &klaida)
00138 {
00139     std::cerr << klaida.what() << "\n";
00140     return 1;
00141 }
00142 return 0;
00143 }
00144
00145 void failoApdorojimas(const std::string &failoPavadinimas, int rezervas, int &ndKiekis, bool medianos,
00146 int rPasirinkimas, bool failas)
00147 {
00148     Timer t;
00149     std::vector<Studentas> studentai = skaitymasIsFailo<std::vector<Studentas>>(failoPavadinimas,
00150     ndKiekis, rezervas);
00151     double skaitymoTrukme = t.elapsed(); // Skirtumas (s)
00152     /* t.reset();
00153     skaiciavimas(studentai, medianos, ndKiekis);
00154     double skaiciavimoTrukme = t.elapsed(); */
00155     t.reset();
00156     rusiavimasSkirstymas(studentai, rPasirinkimas, medianos);
00157     double rusiavimoTrukme = t.elapsed();
00158     t.reset();
00159     isvestis(studentai, medianos, failas);
00160     double isvedimoTrukme = t.elapsed(); // Skirtumas (s)
00161     std::cout << "Failo nuskaitymas į studentai vektorių užtruko: " << skaitymoTrukme << " s\n";
00162     /* std::cout << "Rezultatų skaičiavimas užtruko: " << skaiciavimoTrukme << " s\n"; */
00163     std::cout << "Duomenų rūšiavimas pagal pasirinktą parametą užtruko: " << rusiavimoTrukme << " s\n";
00164     std::cout << "Studentų išvedimas užtruko: " << isvedimoTrukme << " s\n";
00165     std::cout << "Bendra trukmė: " << skaitymoTrukme + /* skaiciavimoTrukme + */ rusiavimoTrukme +
00166     isvedimoTrukme << " s\n";
00167 }

```

8.24 vector/main.h Failo Nuoroda

```

#include "investis.h"
#include "isvestis.h"
#include "apdorojimas.h"
#include "random.h"
#include "Timer.h"
#include "test.h"

```

```
#include "main.tpp"
#include <deque>
#include <list>
```

Funkcijos

- void [failoApdorojimas](#) (const std::string &failoPavadinimas, int rezervas, int &ndKiekis, bool medianos, int rPasirinkimas, bool failas)

Apdoroja failą: nuskaityto, surūšiuoja, išveda.

8.24.1 Funkcijos Dokumentacija

8.24.1.1 failoApdorojimas()

```
void failoApdorojimas (
    const std::string & failoPavadinimas,
    int rezervas,
    int & ndKiekis,
    bool medianos,
    int rPasirinkimas,
    bool failas)
```

Apdoroja failą: nuskaityto, surūšiuoja, išveda.

Parametrai

<i>failoPavadinimas</i>	Failo, iš kurio skaityti, pavadinimas.
<i>rezervas</i>	Rezervavimo dydis konteneriui (jei žinomas).
<i>ndKiekis</i>	Namų darbų skaičius.
<i>medianos</i>	Ar naudoti medianą galutiniam pažymiui.
<i>rPasirinkimas</i>	Rūšiavimo būdas.
<i>failas</i>	Ar išvesti į failą (true) ar į ekraną (false).

Apibrėžimas failo [main.cpp](#) eilutėje 145.

8.25 main.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef MAIN_H
00002 #define MAIN_H
00003
00004 //Header file includes
00005 #include "investis.h"
00006 #include "investis.h"
00007 #include "apdorojimas.h"
00008 #include "random.h"
00009 #include "Timer.h"
00010 #include "test.h"
00011 #include "main.tpp"
00012 #include <deque>
00013 #include <list>
```

```

00014
00015 #ifdef _WIN32 // naudojame preprocesorių, kad kompiliatorius, naudojant Windows, pridėtų windows.h
        antraščių failą, kad vėliau galėtume pakeisti terminalo išvesties ir įvesties užkodavimą į UTF-8
00016 #include <windows.h> // windows antraščių failas
00017 #endif
00018
00028 void failoApdorojimas(const std::string &failoPavadinimas, int rezervas, int &ndKiekis, bool medianos,
        int rPasirinkimas, bool failas);
00029
00030 #endif

```

8.26 vector/main.cpp Failo Nuoroda

```

#include "main.h"
#include "apdorojimas.cpp"
#include "isvestis.cpp"

```

Funkcijos

- `template<class Konteineris>`
`void skirstymoPasirinkimas` (int sPasirinkimas, Konteineris &studentai, Konteineris &galvociai, Konteineris &vargsiukai)
Pasirenka skirstymo strategiją.
- `template<class Konteineris>`
`void failoTestavimas` (const std::string &failoPavadinimas, int rezervas, int tPasirinkimas, int sPasirinkimas, int &ndKiekis, bool medianos)
Atlieka failo testavimą su duotu konteinerio tipu.
- `template<class Konteineris>`
`void skirstymoPasirinkimas` (int sPasirinkimas, Konteineris &studentai, Konteineris &galvociai, Konteineris &vargsiukai, bool medianos, int ndKiekis=0)
Įgyvendina skirstymo strategijos pasirinkimą.

8.26.1 Funkcijos Dokumentacija

8.26.1.1 failoTestavimas()

```

template<class Konteineris>
void failoTestavimas (
    const std::string & failoPavadinimas,
    int rezervas,
    int tPasirinkimas,
    int sPasirinkimas,
    int & ndKiekis,
    bool medianos)

```

Atlieka failo testavimą su duotu konteinerio tipu.

Template Parameters

<i>Konteineris</i>	(vector, deque, list).
--------------------	------------------------

Parametrai

<i>failoPavadinimas</i>	Failo pavadinimas.
-------------------------	--------------------

<i>rezervas</i>	Rezervavimo dydis.
<i>tPasirinkimas</i>	Kiek kartų testuoti.
<i>sPasirinkimas</i>	Skirstymo strategija.
<i>ndKiekis</i>	Namų darbų skaičius.
<i>medianos</i>	Ar naudoti medianą galutiniam pažymiui.

Apibrėžimas failo [main.hpp](#) eilutėje 29.

8.26.1.2 skirstymoPasirinkimas() [1/2]

```
template<class Konteineris>
void skirstymoPasirinkimas (
    int sPasirinkimas,
    Konteineris & studentai,
    Konteineris & galvociiai,
    Konteineris & vargsiukai)
```

Pasirenka skirstymo strategiją.

Template Parameters

<i>Konteineris</i>	(list, vector, deque).
--------------------	------------------------

Parametrai

<i>sPasirinkimas</i>	Strategijos numeris (0-3).
<i>studentai</i>	Studentų konteineris.
<i>galvociiai</i>	Konteineris studentams, kurių galutinis ≥ 5 .
<i>vargsiukai</i>	Konteineris studentams, kurių galutinis < 5 .
<i>medianos</i>	Ar naudoti medianą galutiniam pažymiui.
<i>ndKiekis</i>	Namų darbų skaičius.

8.26.1.3 skirstymoPasirinkimas() [2/2]

```
template<class Konteineris>
void skirstymoPasirinkimas (
    int sPasirinkimas,
    Konteineris & studentai,
    Konteineris & galvociiai,
    Konteineris & vargsiukai,
    bool medianos,
    int ndKiekis = 0)
```

Igyvendina skirstymo strategijos pasirinkimą.

Template Parameters

<i>Konteineris</i>	(vector, deque, list).
--------------------	------------------------

Apibrėžimas failo [main.hpp](#) eilutėje 92.

8.27 main.cpp

Eiti į šio failo dokumentaciją.

```

00001 #include "main.h"
00002 #include "apdorojimas.cpp"
00003 #include "isvestis.cpp"
00004
00015 template <class Konteineris>
00016 void skirstymoPasirinkimas(int sPasirinkimas, Konteineris &studentai, Konteineris &galvociiai,
    Konteineris &vargsiukai);
00017
00028 template <class Konteineris>
00029 void failoTestavimas(const std::string &failoPavadinimas, int rezervas, int tPasirinkimas, int
    sPasirinkimas, int &ndKiekis, bool medianos)
00030 {
00031     double skaitymoTrukme = 0;
00032     double skaiciavimoTrukme = 0;
00033     double rusiavimoTrukme = 0;
00034     double skirstymoTrukme = 0;
00035     double isvedimoTrukme1 = 0;
00036     double isvedimoTrukme2 = 0;
00037     ndKiekis = 0;
00038     Konteineris studentai = skaitymasIsFailo<Konteineris>(failoPavadinimas, ndKiekis, rezervas); //
nuskaitymas
00039     /* skaiciavimas<Konteineris>(studentai, medianos, ndKiekis); */ // rezultatu apsiskaiciavimas
    pries rikiavima
00040     rusiavimasSkirstymas(studentai, 5, medianos, ndKiekis); // rusiavimas didejanciai
00041     Konteineris vargsiukai;
00042     Konteineris galvociiai;
00043     if constexpr(requires(Konteineris konteineris){konteineris.reserve(0);}){
00044         galvociiai.reserve(studentai.size());
00045         vargsiukai.reserve(studentai.size());
00046     }
00047     skirstymoPasirinkimas(sPasirinkimas, studentai, galvociiai, vargsiukai, medianos, ndKiekis);
00048     /* isvestis(galvociiai, medianos, true, "galvociiai.txt");
00049     isvestis(vargsiukai, medianos, true, "vargsiukai.txt"); */
00050     for(int i = 0; i < tPasirinkimas; i++)
00051     {
00052         ndKiekis = 0;
00053         Timer t;
00054         Konteineris studentai = skaitymasIsFailo<Konteineris>(failoPavadinimas, ndKiekis, rezervas);
// nuskaitymas
00055         skaitymoTrukme += t.elapsed(); // Skirtumas (s)
00056         /* t.reset();
00057         skaiciavimas<Konteineris>(studentai, medianos, ndKiekis); // rezultatu apsiskaiciavimas pries
    rikiavima
00058         skaiciavimoTrukme += t.elapsed(); */
00059         t.reset();
00060         rusiavimasSkirstymas(studentai, 5, medianos, ndKiekis); // rusiavimas didejanciai
00061         rusiavimoTrukme += t.elapsed();
00062         t.reset();
00063         Konteineris vargsiukai;
00064         Konteineris galvociiai;
00065         if constexpr(requires(Konteineris konteineris){konteineris.reserve(0);}){
00066             galvociiai.reserve(studentai.size());
00067             vargsiukai.reserve(studentai.size());
00068         }
00069         skirstymoPasirinkimas(sPasirinkimas, studentai, galvociiai, vargsiukai, medianos, ndKiekis);
00070         skirstymoTrukme += t.elapsed();
00071         t.reset();
00072         isvestis(galvociiai, medianos, true, "galvociiai.txt");
00073         isvedimoTrukme1 += t.elapsed(); // Skirtumas (s)
00074         t.reset();
00075         isvestis(vargsiukai, medianos, true, "vargsiukai.txt");
00076         isvedimoTrukme2 += t.elapsed(); // Skirtumas (s)
00077     }
00078     std::cout << "Failo nuskaitymas į studentai konteinerį vidutiniškai užtruko: " <<
    skaitymoTrukme/tPasirinkimas << " s\n";
00079     /* std::cout << "Rezultatų skaičiavimas vidutiniškai užtruko: " << skaiciavimoTrukme/tPasirinkimas <<
    " s\n"; */
00080     std::cout << "Duomenų rūšiavimas didėjančiai vidutiniškai užtruko: " <<
    rusiavimoTrukme/tPasirinkimas << " s\n";
00081     std::cout << "Studentų skirstymas pagal pažymius vidutiniškai užtruko: " <<
    skirstymoTrukme/tPasirinkimas << " s\n";
00082     std::cout << "Studentų išvedimas į galvociiai.txt vidutiniškai užtruko: " <<
    isvedimoTrukme1/tPasirinkimas << " s\n";
00083     std::cout << "Studentų išvedimas į vargsiukai.txt vidutiniškai užtruko: " <<
    isvedimoTrukme2/tPasirinkimas << " s\n";
00084     std::cout << "Bendra trukmė: " << skaitymoTrukme + skaiciavimoTrukme + rusiavimoTrukme +
    skirstymoTrukme + isvedimoTrukme1 + isvedimoTrukme2 << " s\n";
00085 }
00086
00091 template <class Konteineris>
00092 void skirstymoPasirinkimas(int sPasirinkimas, Konteineris &studentai, Konteineris &galvociiai,
    Konteineris &vargsiukai, bool medianos, int ndKiekis = 0)

```



```

00093 {
00094     switch (sPasirinkimas)
00095     {
00096         case 0:
00097         {
00098             skirstymas(studentai, galvociai, vargsiukai, medianos, ndKiekis);
00099             break;
00100         }
00101         case 1:
00102         {
00103             skirstymasStrat1(studentai, galvociai, vargsiukai, medianos, ndKiekis);
00104             break;
00105         }
00106         case 2:
00107         {
00108             skirstymasStrat2(studentai, vargsiukai, medianos, ndKiekis);
00109             break;
00110         }
00111         case 3:
00112         {
00113             skirstymasStrat3(studentai, vargsiukai, medianos, ndKiekis);
00114             break;
00115         }
00116         default:
00117         {
00118             break;
00119         }
00120     }
00121 }

```

8.28 vector/random.cpp Failo Nuoroda

```

#include "random.h"
#include "investis.h"

```

Funkcijos

- `std::vector< Studentas > ivesitiStudentusRandom (int pasirinkimas)`
Generuoja studentų sąrašą su atsitiktiniais duomenimis.
- `std::mt19937 & rng ()`
Gražina statinį Mersenne Twister atsitiktinių skaičių generatorių.

8.28.1 Funkcijos Dokumentacija

8.28.1.1 ivesitiStudentusRandom()

```

std::vector< Studentas > ivesitiStudentusRandom (
    int pasirinkimas)

```

Generuoja studentų sąrašą su atsitiktiniais duomenimis.

Parametrai

<i>pasirinkimas</i>	2 – generuoti tik pažymius (vardai/pavardės įvedami ranka), 3 – generuoti viską.
---------------------	--

Gražina

Vektorius su sugeneruotais studentais.

Apibrėžimas failo `random.cpp` eilutėje 4.

8.28.1.2 rng()

```
std::mt19937 & rng ()
```

Gražina statinį Mersenne Twister atsitiktinių skaičių generatorių.

Gražina

Nuoroda į generatorių.

Apibrėžimas failo [random.cpp](#) eilutėje 40.

8.29 random.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "random.h"
00002 #include "investis.h"
00003
00004 std::vector<Studentas> ivestiStudentusRandom(int pasirinkimas)
00005 {
00006     std::vector<Studentas> studentai;
00007     switch(pasirinkimas)
00008     {
00009         case 2:
00010         {
00011             Studentas A;
00012             while(A.readSemiRandom())
00013             {
00014                 studentai.push_back(std::move(A));
00015                 A = Studentas{};
00016             }
00017             break;
00018         }
00019         case 3:
00020         {
00021             int studKiekis = gautiSkaiciu("Įveskite norimą generuoti studentų kiekį: ", 0, 100);
00022             studentai.reserve(studKiekis);
00023             for (int i = 0; i < studKiekis; i++)
00024             {
00025                 Studentas A;
00026                 int ndKiekis = gautiSkaiciu("Įveskite norimą generuoti namų darbų rezultatų kiekį: ",
00027 0, 100);
00028                 A.readRandom(ndKiekis);
00029                 studentai.push_back(std::move(A));
00030             }
00031             break;
00032         }
00033         default:
00034         {
00035             std::cout << "ivestiStudentusRandom default atvejis" << std::endl;
00036         }
00037     }
00038     return studentai;
00039 }
00040 std::mt19937& rng() {
00041     static std::mt19937 rng(std::random_device{}());
00042     return rng;
00043 }
```

8.30 vector/random.h Failo Nuoroda

```
#include "studentas.h"
#include <random>
```

Funkcijos

- `std::vector< Studentas > ivesitiStudentusRandom` (int pasirinkimas)
Generuoja studentų sąrašą su atsitiktiniais duomenimis.
- `std::mt19937 & rng` ()
Gražina statinį Mersenne Twister atsitiktinių skaičių generatorių.

8.30.1 Funkcijos Dokumentacija

8.30.1.1 ivesitiStudentusRandom()

```
std::vector< Studentas > ivesitiStudentusRandom (
    int pasirinkimas)
```

Generuoja studentų sąrašą su atsitiktiniais duomenimis.

Parametrai

<i>pasirinkimas</i>	2 – generuoti tik pažymius (vardai/pavardės įvedami ranka), 3 – generuoti viską.
---------------------	--

Gražina

Vektorius su sugeneruotais studentais.

Apibrėžimas failo [random.cpp](#) eilutėje 4.

8.30.1.2 rng()

```
std::mt19937 & rng ()
```

Gražina statinį Mersenne Twister atsitiktinių skaičių generatorių.

Gražina

Nuoroda į generatorių.

Apibrėžimas failo [random.cpp](#) eilutėje 40.

8.31 random.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef RANDOM_H
00002 #define RANDOM_H
00003 #include "studentas.h"
00004 #include <random>
00005
00012 std::vector<Studentas> ivesitiStudentusRandom(int pasirinkimas);
00013
00018 std::mt19937& rng();
00019
00020 #endif
```

8.32 vector/studentas.cpp Failo Nuoroda

```
#include "studentas.h"
#include "investis.h"
#include "random.h"
#include <algorithm>
#include <numeric>
#include <iostream>
#include <fstream>
#include <random>
#include <chrono>
#include <utility>
```

Funkcijos

- std::istream & [operator>>](#) (std::istream &is, [Studentas](#) &A)
- std::ostream & [operator<<](#) (std::ostream &out, const [Studentas](#) &A)

8.32.1 Funkcijos Dokumentacija

8.32.1.1 operator<<()

```
std::ostream & operator<< (
    std::ostream & out,
    const Studentas & A)
```

Apibrėžimas failo [studentas.cpp](#) eilutėje 261.

8.32.1.2 operator>>()

```
std::istream & operator>> (
    std::istream & is,
    Studentas & A)
```

Apibrėžimas failo [studentas.cpp](#) eilutėje 255.

8.33 studentas.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "studentas.h"
00002 #include "investis.h"
00003 #include "random.h"
00004 #include <algorithm>
00005 #include <numeric>
00006 #include <iostream>
00007 #include <fstream>
00008 #include <random>
00009 #include <chrono>
00010 #include <utility>
00011
00012 void Studentas::addNd(int paz)
00013 {
```

```

00014     nd.push_back(paz);
00015 }
00016
00017 void Studentas::resizeNd(int n, int skaicius)
00018 {
00019     nd.resize(n, skaicius);
00020 };
00021
00022 double Studentas::galutinis(bool medianos, int ndKiekis) const
00023 {
00024     const int max = (ndKiekis != 0) ? ndKiekis : maxNdKiekis;
00025     if(nd.empty()) return 0.6*rez;
00026     if(medianos)
00027     {
00028         std::vector<int> ND = nd;
00029         if(ND.size() < max) ND.resize(max, 0);
00030         std::sort(ND.begin(), ND.end());
00031         const int vidurys = max / 2;
00032         std::nth_element(ND.begin(), ND.begin() + vidurys, ND.end());
00033         double med;
00034         if(max % 2 == 0)
00035         {
00036             med = (*std::max_element(ND.begin(), ND.begin() + vidurys) + ND[vidurys]) / 2.0;
00037         }
00038         else
00039         {
00040             med = ND[vidurys];
00041         }
00042         return 0.4 * med + 0.6 * rez;
00043     }
00044     else
00045     {
00046         int sum = std::accumulate(nd.begin(), nd.end(), 0);
00047
00048         double vid = (double)sum / max;
00049         return 0.4 * vid + 0.6 * rez;
00050     }
00051 }
00052
00053 std::istream& Studentas::readStudentStream(std::istream& is, int ndKiekis)
00054 {
00055     if(!(is >> vardas >> pavarde)) return is;
00056     int paz;
00057     if(ndKiekis == 0)
00058     {
00059         std::vector<int> skaiciai;
00060         while(is >> paz)
00061         {
00062             skaiciai.push_back(paz);
00063         }
00064         if(skaiciai.empty()){
00065             throw std::runtime_error("Klaida: įvesty nerasta nei namų darbų, nei egzamino pažymių");
00066         }
00067         rez = skaiciai.back();
00068         skaiciai.pop_back();
00069         nd = skaiciai;
00070     }
00071     else
00072     {
00073         nd.clear();
00074         nd.reserve(ndKiekis);
00075         for(int i = 0; i < ndKiekis; i++)
00076         {
00077             if(!(is >> paz)) throw std::runtime_error("Klaida: netinkami pažymiai faile.");
00078             nd.push_back(paz);
00079         }
00080         if(!(is >> rez)) throw std::runtime_error("Klaida: netinkamas egzamino rezultatas faile.");
00081     }
00082     return is;
00083 }
00084
00085 std::istream& Studentas::readStudentConsole(std::istream& is){
00086     std::string eilute;
00087     std::cout << "Įveskite studento vardą bei pavardę (ENTER - nutraukti įvedimą): ";
00088
00089     // perskaito eilute ir jei perskaitymas nesekmingas, tai žiuri ar std::cin.eof, jei taip, tai
    programa uzbaiigiama, jei ne, tai išvalo įvesties stream'o veliaveles ir vel prasoma ivesti
00090     if (!std::getline(std::cin, eilute))
00091     {
00092         cinEOFgaudymas();
00093         return is;
00094     } // jei enter - iseina
00095     if(eilute.empty())
00096     {
00097         is.setstate(std::ios::failbit);
00098         return is;
00099     }

```

```

00100     while(!studentoVardoPavardesIvestis(eilute))
00101     {
00102         std::cout << "Įveskite studento vardą bei pavardę (ENTER - baigti): ";
00103         if(!std::getline(std::cin, eilute) || eilute.empty()) return is;
00104     }
00105     namuDarbuRezultatuIvestis();
00106     egzaminoRezultatoIvestis();
00107     return is;
00108 }
00109
00110 bool Studentas::studentoVardoPavardesIvestis(const std::string& eilute)
00111 {
00112     std::istringstream iss(eilute);
00113     if (!(iss >> vardas >> pavarde))
00114     {
00115         std::cout << "Įveskite vardą ir pavardę (du žodžiai)!\n";
00116         return false;
00117     }
00118     // tikriname ar po vardo ir pavardės yra dar žodžių
00119     std::string ekstra;
00120     if (iss >> ekstra)
00121     {
00122         std::cout << "Įvesta per daug žodžių -- reikia tik vardo ir pavardės!\n";
00123         return false;
00124     }
00125     return true;
00126 }
00127
00128 void Studentas::namuDarbuRezultatuIvestis()
00129 {
00130     std::cout << "Įveskite " << maxNdKiekis << " namų darbų rezultatų." << std::endl;
00131     while (nd.size() < maxNdKiekis)
00132     {
00133         int balas = gautiSkaiciu("Įveskite namų darbų rezultatą nuo 1 iki 10 (ENTER - baigti): ", 1,
00134 10, true);
00135         if (balas == -1) break;
00136         nd.push_back(balas);
00137     }
00138     if(nd.size() == maxNdKiekis)
00139     {
00140         std::cout << "Įvestas didžiausias namų darbų rezultatų kiekis" << std::endl;
00141     }
00142 }
00143
00144 void Studentas::egzaminoRezultatoIvestis()
00145 {
00146     rez = gautiSkaiciu("Įveskite egzamino rezultatą (1-10): ", 1, 10);
00147 }
00148
00149 bool Studentas::readSemiRandom()
00150 {
00151     std::string eilute;
00152     std::cout << "Įveskite studento vardą bei pavardę (ENTER - nutraukti įvedimą): ";
00153
00154     // perskaito eilute ir jei perskaitymas nesekmingas, tai žiuri ar std::cin.eof, jei taip, tai
    programa uzbaiigiama, jei ne, tai isvalo ivesties stream'o veliaveles ir vel prasoma ivesti
00155     if (!std::getline(std::cin, eilute))
00156     {
00157         cinEOFgaudymas();
00158         return false;
00159     } // jei enter - iseina
00160     if(eilute.empty())
00161     {
00162         return false;
00163     }
00164     while(!studentoVardoPavardesIvestis(eilute))
00165     {
00166         std::cout << "Įveskite studento vardą bei pavardę (ENTER - baigti): ";
00167         if(!std::getline(std::cin, eilute) || eilute.empty()) return false;
00168     }
00169     ndRandom(gautiSkaiciu("Įveskite norimą generuoti namų darbų rezultatų kiekį: ", 0, 100));
00170     egzRandom();
00171     return true;
00172 }
00173
00174 void Studentas::readRandom(int ndKiekis)
00175 {
00176     genVardaPavarde();
00177     ndRandom(ndKiekis);
00178     egzRandom();
00179 }
00180
00181 void Studentas::genVardaPavarde()
00182 {
00183     std::uniform_int_distribution<int> dist(0, 9);
00184

```

```

00185     static const std::vector<std::string> vardai = {
00186         "Irma", "Alma", "Irena", "Egle", "Jolanta",
00187         "Petras", "Jonas", "Ignas", "Darius", "Simas"
00188     };
00189     static const std::vector<std::string> pavardes_m = {
00190         "Pavardaite1", "Pavardaite2", "Pavardaite3", "Pavardaite4", "Pavardaite5",
00191         "Pavardaite6", "Pavardaite7", "Pavardaite8", "Pavardaite9", "Pavardaite10"
00192     };
00193     static const std::vector<std::string> pavardes_v = {
00194         "Pavardenis1", "Pavardenis2", "Pavardenis3", "Pavardenis4", "Pavardenis5",
00195         "Pavardenis6", "Pavardenis7", "Pavardenis8", "Pavardenis9", "Pavardenis10"
00196     };
00197
00198     vardas = vardai[dist(rng())];
00199     // paskutinis simbolis 's' = vyras
00200     pavarde = (*vardas.rbegin() == 's') ? pavardes_v[dist(rng())] : pavardes_m[dist(rng())];
00201 }
00202
00203 void Studentas::ndRandom(int ndKiekis)
00204 {
00205     std::uniform_int_distribution<int> dist(1, 10);
00206     for (int i = 0; i < ndKiekis; i++)
00207         nd.push_back(dist(rng()));
00208 }
00209
00210 void Studentas::egzRandom()
00211 {
00212     std::uniform_int_distribution<int> dist(1, 10);
00213     rez = dist(rng());
00214 }
00215
00216 Studentas::~Studentas()
00217 {
00218     nd.clear();
00219     rez = 0;
00220 }
00221
00222 Studentas::Studentas(const Studentas& kitas)
00223     : Zmogus(kitas),
00224     nd(kitas.nd),
00225     rez(kitas.rez)
00226 {}
00227
00228 Studentas& Studentas::operator=(const Studentas& kitas)
00229 {
00230     if (this != &kitas)
00231     {
00232         Zmogus::operator=(kitas);
00233         nd = kitas.nd;
00234         rez = kitas.rez;
00235     }
00236     return *this;
00237 }
00238
00239 Studentas::Studentas(Studentas&& kitas) noexcept
00240     : Zmogus(std::move(kitas)),
00241     nd(std::move(kitas.nd)),
00242     rez(std::exchange(kitas.rez, 0))
00243 {}
00244
00245 Studentas& Studentas::operator=(Studentas&& kitas) noexcept
00246 {
00247     if (this != &kitas) {
00248         Zmogus::operator=(std::move(kitas));
00249         nd = std::move(kitas.nd);
00250         rez = std::exchange(kitas.rez, 0);
00251     }
00252     return *this;
00253 }
00254
00255 std::istream& operator>(std::istream& is, Studentas &A)
00256 {
00257     A.read(is);
00258     return is;
00259 }
00260
00261 std::ostream& operator<(std::ostream& out, const Studentas &A) {
00262     out << A.getVardas() << " " << A.getPavarde() << " ";
00263     for (int X : A.getNd()) out << X << " ";
00264     out << A.getRez();
00265     return out;
00266 }
00267
00268 void Studentas::read(std::istream& is, int ndKiekis) {
00269     if (&is == &std::cin)
00270     {
00271         readStudentConsole(is);

```

```
00272     }
00273     else
00274     {
00275         readStudentStream(is, ndKiekis);
00276     }
00277 }
```

8.34 vector/studentas.h Failo Nuoroda

```
#include "zmogus.h"
#include <string>
#include <vector>
#include <istream>
#include <ostream>
```

Klasės

- class [Studentas](#)

Klasė, sukurianti studento objektą su pažymiais.

Kintamieji

- constexpr int [maxNdKiekis](#) = 10

Maksimalus namų darbų kiekis (naudojamas jei nenurodyta kitaip).

8.34.1 Kintamojo Dokumentacija

8.34.1.1 maxNdKiekis

```
int maxNdKiekis = 10 [constexpr]
```

Maksimalus namų darbų kiekis (naudojamas jei nenurodyta kitaip).

Apibrėžimas failo [studentas.h](#) eilutėje 10.

8.35 studentas.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #ifndef STUDENTAS_H
00002 #define STUDENTAS_H
00003
00004 #include "zmogus.h"
00005 #include <string>
00006 #include <vector>
00007 #include <istream>
00008 #include <ostream>
00009
00010 constexpr int maxNdKiekis = 10;
00011
00019 class Studentas : public Zmogus {
00020     std::vector<int> nd;
00021     int rez;
```



```

00022
00028     bool studentoVardoPavardesIvestis(const std::string& eilute);
00029
00031     void namuDarbuRezultatuIvestis();
00032
00034     void egzaminoRezultatuIvestis();
00035
00037     void genVardaPavarde();
00038
00043     void ndRandom(int ndKiekis);
00044
00046     void egzRandom();
00047
00048 public:
00049     // constructors
00051     Studentas() : Zmogus(), nd(), rez(0){};
00052
00060     Studentas(std::string v, std::string p, std::vector<int> n, int r)
00061         : Zmogus(std::move(v), std::move(p)), nd(std::move(n)), rez(r) {};
00062
00068     Studentas(std::istream& is, int ndKiekis = 0){ read(is, ndKiekis); };
00069
00071     Studentas(const Studentas &kitas);
00072
00074     Studentas& operator=(const Studentas&);
00075
00077     Studentas(Studentas&&) noexcept;
00078
00080     Studentas& operator=(Studentas&&) noexcept;
00081
00082     // getters
00084     const std::vector<int> &getNd() const { return nd; };
00085
00087     int getRez() const { return rez; };
00088
00089     // setters
00094     void setNd(std::vector<int> n) { nd = std::move(n); }
00095
00100     void setRez(int r) { rez = r; }
00101
00108     std::istream& readStudentStream(std::istream& is, int ndKiekis);
00109
00115     std::istream& readStudentConsole(std::istream& is);
00116
00122     void read(std::istream& is, int ndKiekis = 0);
00123
00128     bool readSemiRandom();
00129
00134     void readRandom(int ndKiekis);
00135
00140     void addNd(int paz);
00141
00147     void resizeNd(int n, int skaicius = 0);
00148
00155     double galutinis(bool medianos, int ndKiekis = 0) const;
00156
00158     ~Studentas();
00159
00161     friend std::istream& operator>>(std::istream& in, Studentas &A);
00162
00164     friend std::ostream& operator<<(std::ostream& out, const Studentas &A);
00165 };
00166
00167 #endif // STUDENTAS_H

```

8.36 vector/test.cpp Failo Nuoroda

```

#include "test.h"
#include <memory>

```

Funkcijos

- void `testas` ()

Paleidžia visus `Studentas` klasės testus.

- void `testDefaultKonstruktorius ()`
Testuoja numatytąjį konstruktorių.
- void `testStreamKonstruktorius ()`
Testuoja konstruktorių su srautu.
- void `testCopyKonstruktorius ()`
Testuoja kopijavimo konstruktorių.
- void `testCopyAssignmentOperator ()`
Testuoja kopijavimo priskyrimo operatorių.
- void `testMoveKonstruktorius ()`
Testuoja perkėlimo konstruktorių.
- void `testMoveAssignmentOperator ()`
Testuoja perkėlimo priskyrimo operatorių.
- void `testDestructor ()`
Testuoja destruktorių.
- void `testInputOperator ()`
Testuoja įvesties operatorių (>>).
- void `testOutputOperator ()`
Testuoja išvesties operatorių (<<).
- void `testoRezultatas (const std::string &testoPavadinimas, bool passed)`
Išveda testo rezultatą į konsolę.

8.36.1 Funkcijos Dokumentacija

8.36.1.1 testas()

```
void testas ()
```

Paleidžia visus `Studentas` klasės testus.

Apibrėžimas failo `test.cpp` eilutėje 4.

8.36.1.2 testCopyAssignmentOperator()

```
void testCopyAssignmentOperator ()
```

Testuoja kopijavimo priskyrimo operatorių.

Apibrėžimas failo `test.cpp` eilutėje 42.

8.36.1.3 testCopyKonstruktorius()

```
void testCopyKonstruktorius ()
```

Testuoja kopijavimo konstruktorių.

Apibrėžimas failo `test.cpp` eilutėje 34.

8.36.1.4 testDefaultKonstruktorius()

```
void testDefaultKonstruktorius ()
```

Testuoja numatytąjį konstruktorių.

Apibrėžimas failo [test.cpp](#) eilutėje 18.

8.36.1.5 testDestructor()

```
void testDestructor ()
```

Testuoja destruktorių.

Apibrėžimas failo [test.cpp](#) eilutėje 68.

8.36.1.6 testInputOperator()

```
void testInputOperator ()
```

Testuoja įvesties operatorių (>>).

Apibrėžimas failo [test.cpp](#) eilutėje 78.

8.36.1.7 testMoveAssignmentOperator()

```
void testMoveAssignmentOperator ()
```

Testuoja perkėlimo priskyrimo operatorių.

Apibrėžimas failo [test.cpp](#) eilutėje 59.

8.36.1.8 testMoveKonstruktorius()

```
void testMoveKonstruktorius ()
```

Testuoja perkėlimo konstruktorių.

Apibrėžimas failo [test.cpp](#) eilutėje 51.

8.36.1.9 testoRezultatas()

```
void testoRezultatas (  
    const std::string & testName,  
    bool passed)
```

Išveda testo rezultatą į konsolę.

Parametrai

<i>testName</i>	Testo pavadinimas.
-----------------	--------------------

<i>passed</i>	Ar testas pavyko (true) ar ne (false).
---------------	--

Apibrėžimas failo [test.cpp](#) eilutėje 98.

8.36.1.10 testOutputOperator()

```
void testOutputOperator ()
```

Testuoja išvesties operatorių (<<).

Apibrėžimas failo [test.cpp](#) eilutėje 88.

8.36.1.11 testStreamKonstruktorius()

```
void testStreamKonstruktorius ()
```

Testuoja konstruktorių su srautu.

Apibrėžimas failo [test.cpp](#) eilutėje 25.

8.37 test.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "test.h"
00002 #include <memory>
00003
00004 void testas()
00005 {
00006     std::cout << "Studento klasės testas\n\n";
00007     testDefaultKonstruktorius();
00008     testStreamKonstruktorius();
00009     testCopyKonstruktorius();
00010     testCopyAssignmentOperator();
00011     testMoveKonstruktorius();
00012     testMoveAssignmentOperator();
00013     testDestructor();
00014     testInputOperator();
00015     testOutputOperator();
00016 }
00017
00018 void testDefaultKonstruktorius()
00019 {
00020     Studentas A;
00021     bool testas = (A.getVardas().empty() && A.getPavarde().empty() && A.getNd().empty() && A.getRez()
00022 == 0);
00023     testoRezultatas("Default konstruktorius", testas);
00024 }
00025 void testStreamKonstruktorius()
00026 {
00027     std::istringstream iss("Jonas Jonaitis 1 2 10");
00028     Studentas C(iss, 2);
00029     bool testas = (C.getVardas()=="Jonas" && C.getPavarde()=="Jonaitis" && C.getNd().at(0) == 1 &&
00030 C.getNd().at(1) == 2 && C.getRez() == 10);
00031     std::cout << "Turėtų išvesti 'Jonas Jonaitis 1 2 10':\n" << C << std::endl;
00032     testoRezultatas("Stream įvesties konstruktorius", testas);
00033 }
00034 void testCopyKonstruktorius()
00035 {
00036     Studentas A("A", "AAA", {1,2}, 10);
00037     Studentas B(A);
00038     bool testas = (B.getVardas()=="A" && B.getPavarde()=="AAA" && B.getNd().at(0) == 1 &&
00039 B.getNd().at(1) == 2 && B.getRez() == 10);
```

```

00039     testoRezultatas("Copy konstruktorius", testas);
00040 }
00041
00042 void testCopyAssignmentOperator()
00043 {
00044     Studentas A("A", "AAA", {1,2}, 10);
00045     Studentas B;
00046     B = A;
00047     bool testas = (B.getVardas()=="A" && B.getPavarde()=="AAA" && B.getNd().at(0) == 1 &&
B.getNd().at(1) == 2 && B.getRez() == 10);
00048     testoRezultatas("Copy assignment operatorius", testas);
00049 }
00050
00051 void testMoveKonstruktorius()
00052 {
00053     Studentas A("A", "AAA", {1,2}, 10);
00054     Studentas B(std::move(A));
00055     bool testas = ((B.getVardas()=="A" && B.getPavarde()=="AAA" && B.getNd().at(0) == 1 &&
B.getNd().at(1) == 2 && B.getRez() == 10) && (A.getVardas().empty() && A.getPavarde().empty() &&
A.getNd().empty() && A.getRez() == 0));
00056     testoRezultatas("Move konstruktorius", testas);
00057 }
00058
00059 void testMoveAssignmentOperator()
00060 {
00061     Studentas A("A", "AAA", {1,2}, 10);
00062     Studentas B;
00063     B = std::move(A);
00064     bool testas = ((B.getVardas()=="A" && B.getPavarde()=="AAA" && B.getNd().at(0) == 1 &&
B.getNd().at(1) == 2 && B.getRez() == 10) && (A.getVardas().empty() && A.getPavarde().empty() &&
A.getNd().empty() && A.getRez() == 0));
00065     testoRezultatas("Move assignment operatorius", testas);
00066 }
00067
00068 void testDestructor()
00069 {
00070     std::weak_ptr<Studentas> weak; // "protinga" rodykle, kuri siuo atveju rodo i scope esancia shared
protinga rodykle, taciau weak rodyklei nepriklauso shared A rodykle, t.y. ji nera savininke, tiesiog
rodo i ta pati objekta kaip ir shared rodykle, t.y. i Studenta A, ir jei nerodo i nieka galima
patikrinti su .expired();
00071     {
00072         std::shared_ptr<Studentas> A = std::make_shared<Studentas>("A", "AAA", std::vector<int>{1,2},
10);
00073         weak = A;
00074     }
00075     testoRezultatas("Destruktorius", weak.expired());
00076 }
00077
00078 void testInputOperator()
00079 {
00080     Studentas A;
00081     std::istringstream iss("Jonas Jonaitis 1 2 10");
00082     iss >> A;
00083     std::cout << "Turėty išvesti 'Jonas Jonaitis 1 2 10':\n" << A << std::endl;
00084     bool testas = (A.getVardas()=="Jonas" && A.getPavarde()=="Jonaitis" && A.getNd().at(0) == 1 &&
A.getNd().at(1) == 2 && A.getRez() == 10);
00085     testoRezultatas("Išvesties operatorius", testas);
00086 }
00087
00088 void testOutputOperator()
00089 {
00090     Studentas A("A", "AAA", {1,2}, 10);
00091     std::ostringstream out;
00092     out << A;
00093     std::string isvestis = out.str();
00094     bool testas = (isvestis.find("A") != std::string::npos && isvestis.find("AAA") !=
std::string::npos && isvestis.find("1") != std::string::npos && isvestis.find("2") !=
std::string::npos && isvestis.find("10") != std::string::npos);
00095     testoRezultatas("Išvesties operatorius", testas);
00096 }
00097
00098 void testoRezultatas(const std::string& testoPavadinimas, bool passed)
00099 {
00100     std::cout << (passed ? "PASS " : "FAIL ") << testoPavadinimas << "\n";
00101 }

```

8.38 vector/test.h Failo Nuoroda

```

#include "studentas.h"
#include "investis.h"

```

Funkcijos

- void `testas` ()
Paleidžia visus `Studentas` klasės testus.
- void `testoRezultatas` (const std::string &testName, bool passed)
Išveda testo rezultatą į konsolę.
- void `testDefaultKonstruktorius` ()
Testuoja numatytąjį konstruktorių.
- void `testStreamKonstruktorius` ()
Testuoja konstruktorių su srautu.
- void `testCopyKonstruktorius` ()
Testuoja kopijavimo konstruktorių.
- void `testCopyAssignmentOperator` ()
Testuoja kopijavimo priskyrimo operatorių.
- void `testMoveKonstruktorius` ()
Testuoja perkėlimo konstruktorių.
- void `testMoveAssignmentOperator` ()
Testuoja perkėlimo priskyrimo operatorių.
- void `testDestructor` ()
Testuoja destruktorių.
- void `testInputOperator` ()
Testuoja įvesties operatorių (>>).
- void `testOutputOperator` ()
Testuoja išvesties operatorių (<<).

8.38.1 Funkcijos Dokumentacija

8.38.1.1 `testas()`

```
void testas ()
```

Paleidžia visus `Studentas` klasės testus.

Apibrėžimas failo `test.cpp` eilutėje 4.

8.38.1.2 `testCopyAssignmentOperator()`

```
void testCopyAssignmentOperator ()
```

Testuoja kopijavimo priskyrimo operatorių.

Apibrėžimas failo `test.cpp` eilutėje 42.

8.38.1.3 `testCopyKonstruktorius()`

```
void testCopyKonstruktorius ()
```

Testuoja kopijavimo konstruktorių.

Apibrėžimas failo `test.cpp` eilutėje 34.

8.38.1.4 testDefaultKonstruktorius()

```
void testDefaultKonstruktorius ()
```

Testuoja numatytąjį konstruktorių.

Apibrėžimas failo [test.cpp](#) eilutėje 18.

8.38.1.5 testDestructor()

```
void testDestructor ()
```

Testuoja destruktorių.

Apibrėžimas failo [test.cpp](#) eilutėje 68.

8.38.1.6 testInputOperator()

```
void testInputOperator ()
```

Testuoja įvesties operatorių (>>).

Apibrėžimas failo [test.cpp](#) eilutėje 78.

8.38.1.7 testMoveAssignmentOperator()

```
void testMoveAssignmentOperator ()
```

Testuoja perkėlimo priskyrimo operatorių.

Apibrėžimas failo [test.cpp](#) eilutėje 59.

8.38.1.8 testMoveKonstruktorius()

```
void testMoveKonstruktorius ()
```

Testuoja perkėlimo konstruktorių.

Apibrėžimas failo [test.cpp](#) eilutėje 51.

8.38.1.9 testoRezultatas()

```
void testoRezultatas (  
    const std::string & testName,  
    bool passed)
```

Išveda testo rezultatą į konsolę.

Parametrai

<i>testName</i>	Testo pavadinimas.
-----------------	--------------------

<i>passed</i>	Ar testas pavyko (true) ar ne (false).
---------------	--

Apibrėžimas failo [test.cpp](#) eilutėje 98.

8.38.1.10 testOutputOperator()

```
void testOutputOperator ()
```

Testuoja išvesties operatorių (<<).

Apibrėžimas failo [test.cpp](#) eilutėje 88.

8.38.1.11 testStreamKonstruktorius()

```
void testStreamKonstruktorius ()
```

Testuoja konstruktorių su srautu.

Apibrėžimas failo [test.cpp](#) eilutėje 25.

8.39 test.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #ifndef TEST_H
00002 #define TEST_H
00003
00004 #include "studentas.h"
00005 #include "investis.h"
00006
00010 void testas();
00011
00017 void testoRezultatas(const std::string& testName, bool passed);
00018
00020 void testDefaultKonstruktorius();
00021
00023 void testStreamKonstruktorius();
00024
00026 void testCopyKonstruktorius();
00027
00029 void testCopyAssignmentOperator();
00030
00032 void testMoveKonstruktorius();
00033
00035 void testMoveAssignmentOperator();
00036
00038 void testDestructor();
00039
00041 void testInputOperator();
00042
00044 void testOutputOperator();
00045
00046 #endif
```

8.40 vector/Timer.h Failo Nuoroda

```
#include <chrono>
```


Klasės

- class `Timer`

8.41 Timer.h

Eiti į šio failo dokumentaciją.

```
00001 #include <chrono>
00002 class Timer {
00003     private:
00004         // panaudojame using
00005         using hrClock = std::chrono::high_resolution_clock;
00006         using durationDouble = std::chrono::duration<double>;
00007         std::chrono::time_point<hrClock> start;
00008     public:
00009         Timer() : start{ hrClock::now() } {}
00010         void reset() {
00011             start = hrClock::now();
00012         }
00013         double elapsed() const {
00014             return durationDouble (hrClock::now() - start).count();
00015         }
00016 };
```

8.42 vector/zmogus.cpp Failo Nuoroda

```
#include "zmogus.h"
```

8.43 zmogus.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "zmogus.h"
00002
00003 //copy constructor
00004 Zmogus::Zmogus(const Zmogus &kitas)
00005     : vardas(kitas.vardas),
00006     pavarde(kitas.pavarde)
00007 {}
00008
00009 // copy assignment operator
00010 Zmogus& Zmogus::operator=(const Zmogus& kitas)
00011 {
00012     if(this != &kitas)
00013     {
00014         vardas = kitas.vardas;
00015         pavarde = kitas.pavarde;
00016     }
00017     return *this;
00018 }
00019
00020 // move constructor
00021 Zmogus::Zmogus(Zmogus&& kitas) noexcept
00022     : vardas(std::move(kitas.vardas)),
00023     pavarde(std::move(kitas.pavarde))
00024 {}
00025
00026 // move assignment operator
00027 Zmogus& Zmogus::operator=(Zmogus&& kitas) noexcept
00028 {
00029     if(this != &kitas)
00030     {
00031         vardas = std::move(kitas.vardas);
00032         pavarde = std::move(kitas.pavarde);
00033     }
00034     return *this;
00035 }
00036
00037 Zmogus::~Zmogus() {
00038     vardas.clear();
00039     pavarde.clear();
00040 };
```

8.44 vector/zmogus.h Failo Nuoroda

```
#include <string>
```

Klasės

- class [Zmogus](#)

Abstrakti bazinė klasė, turinti vardą ir pavardę.

8.45 zmogus.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003
00004 #include <string>
00005
00013 class Zmogus {
00014     protected:
00015         std::string vardas;
00016         std::string pavarde;
00017
00018     public:
00020         Zmogus() : vardas(""), pavarde("") {};
00021
00027         Zmogus(const std::string v, const std::string p)
00028             : vardas(std::move(v)), pavarde(std::move(p)) {};
00029
00031         Zmogus(const Zmogus &kitas);
00032
00034         Zmogus& operator=(const Zmogus&);
00035
00037         Zmogus(Zmogus&&) noexcept;
00038
00040         Zmogus& operator=(Zmogus&&) noexcept;
00041
00043         const std::string& getVardas() const { return vardas; }
00044
00046         const std::string& getPavarde() const { return pavarde; }
00047
00052         void setVardas(std::string v) { vardas = std::move(v); }
00053
00058         void setPavarde(std::string p) { pavarde = std::move(p); }
00059
00063         virtual ~Zmogus() = 0;
00064 };
00065
00066 #endif
```

Rodyklė

~Studentas
Studentas, [19](#)
~Zmogus
Zmogus, [28](#)

addNd
Studentas, [19](#)
apdorojimas.cpp
failoGeneravimas, [34](#)
setw, [34](#)
apdorojimas.h
failoGeneravimas, [35](#)
setw, [35](#)
apdorojimas.tpp
rusiavimasPagal, [38](#)
rusiavimasSkirstymas, [38](#)
skirstymas, [38](#)
skirstymasStrat1, [39](#)
skirstymasStrat2, [40](#)
skirstymasStrat3, [40](#)
arTikSkaicius
investis.cpp, [55](#)
investis.h, [58](#)

cinEOFgaudymas
investis.cpp, [55](#)
investis.h, [58](#)

durationDouble
Timer, [25](#)

egzaminoRezultatolvestis
investis.h, [59](#)
Studentas, [19](#)

egzRandom
Studentas, [19](#)

elapsed
Timer, [26](#)

failoApdorojimas
main.cpp, [64](#)
main.h, [67](#)

failoGeneravimas
apdorojimas.cpp, [34](#)
apdorojimas.h, [35](#)

failoPasirinkimas
investis.cpp, [43](#)
investis.h, [48](#)

failoTestavimas
main.tpp, [68](#)

failoUzklausa
investis.cpp, [43](#)
investis.h, [49](#)

galutinis
Studentas, [19](#)

gautiPatvirtinima
investis.cpp, [55](#)
investis.h, [59](#)

gautiSkaiciu
investis.cpp, [55](#)
investis.h, [59](#)

gautiTipoPasirinkima
investis.cpp, [43](#)
investis.h, [49](#)

genVardaPavarde
Studentas, [20](#)

getNd
Studentas, [20](#)

getPavarde
Zmogus, [29](#)

getRez
Studentas, [20](#)

getVardas
Zmogus, [29](#)

gtest.cpp, [31](#)
TEST, [31](#), [32](#)

hrClock
Timer, [25](#)

investis
investis.tpp, [52](#)

investis.cpp
failoPasirinkimas, [43](#)
failoUzklausa, [43](#)
gautiTipoPasirinkima, [43](#)
lietuviskosRaides, [44](#)
medianosUzklausa, [44](#)
menu, [44](#)
ndPasirinkimas, [44](#)
rusiavimoPasirinkimas, [45](#)
strategijosPasirinkimas, [45](#)
studentuPasirinkimas, [45](#)
testavimoPasirinkimas, [45](#)

investis.h
failoPasirinkimas, [48](#)
failoUzklausa, [49](#)
gautiTipoPasirinkima, [49](#)
lietuviskosRaides, [49](#)

- medianosUzklausa, [49](#)
- menu, [50](#)
- ndPasirinkimas, [50](#)
- rusiavimoPasirinkimas, [50](#)
- strategijosPasirinkimas, [50](#)
- studentuPasirinkimas, [51](#)
- testavimoPasirinkimas, [51](#)
- isvestis.hpp
 - isvestis, [52](#)
- investis.cpp
 - arTikSkaicius, [55](#)
 - cinEOFgaudymas, [55](#)
 - gautiPatvirtinima, [55](#)
 - gautiSkaiciu, [55](#)
 - investiStudentus, [56](#)
- investis.h
 - arTikSkaicius, [58](#)
 - cinEOFgaudymas, [58](#)
 - egzaminoRezultatolvestis, [59](#)
 - gautiPatvirtinima, [59](#)
 - gautiSkaiciu, [59](#)
 - investiStudentus, [60](#)
 - namuDarbuRezultatulvestis, [60](#)
 - skaitymas, [60](#)
 - skaitymasIsFailo, [60](#)
 - studentoVardoPavardesIvestis, [61](#)
- investis.hpp
 - skaitymasIsFailo, [62](#)
- investiStudentus
 - investis.cpp, [56](#)
 - investis.h, [60](#)
- investiStudentusRandom
 - random.cpp, [71](#)
 - random.h, [73](#)
- lietuviskosRaides
 - isvestis.cpp, [44](#)
 - isvestis.h, [49](#)
- main
 - main.cpp, [64](#)
- main.cpp
 - failoApdorojimas, [64](#)
 - main, [64](#)
- main.h
 - failoApdorojimas, [67](#)
- main.hpp
 - failoTestavimas, [68](#)
 - skirstymoPasirinkimas, [69](#)
- maxNdKiekis
 - studentas.h, [78](#)
- medianosUzklausa
 - isvestis.cpp, [44](#)
 - isvestis.h, [49](#)
- menu
 - isvestis.cpp, [44](#)
 - isvestis.h, [50](#)
- namuDarbuRezultatulvestis
 - investis.h, [60](#)
 - Studentas, [20](#)
- nd
 - Studentas, [24](#)
- ndPasirinkimas
 - isvestis.cpp, [44](#)
 - isvestis.h, [50](#)
- ndRandom
 - Studentas, [20](#)
- operator<<
 - Studentas, [24](#)
 - studentas.cpp, [74](#)
- operator>>
 - Studentas, [24](#)
 - studentas.cpp, [74](#)
- operator=
 - Studentas, [21](#)
 - Zmogus, [29](#)
- pavarde
 - Zmogus, [30](#)
- Programos naudojimas:, [1](#)
- random.cpp
 - investiStudentusRandom, [71](#)
 - rng, [71](#)
- random.h
 - investiStudentusRandom, [73](#)
 - rng, [73](#)
- read
 - Studentas, [21](#)
- README.md, [33](#)
- readRandom
 - Studentas, [21](#)
- readSemiRandom
 - Studentas, [22](#)
- readStudentConsole
 - Studentas, [22](#)
- readStudentStream
 - Studentas, [22](#)
- reset
 - Timer, [26](#)
- resizeNd
 - Studentas, [23](#)
- rez
 - Studentas, [24](#)
- rng
 - random.cpp, [71](#)
 - random.h, [73](#)
- rusiavimasPagal
 - apdorojimas.hpp, [38](#)
- rusiavimasSkirstymas
 - apdorojimas.hpp, [38](#)
- rusiavimoPasirinkimas
 - isvestis.cpp, [45](#)
 - isvestis.h, [50](#)
- setNd

- Studentas, 23
- setPavarde
 - Zmogus, 29
- setRez
 - Studentas, 23
- setVardas
 - Zmogus, 29
- setw
 - apdorojimas.cpp, 34
 - apdorojimas.h, 35
- skaitymas
 - investis.h, 60
- skaitymasIsFailo
 - investis.h, 60
 - investis.tpp, 62
- skirstymas
 - apdorojimas.tpp, 38
- skirstymasStrat1
 - apdorojimas.tpp, 39
- skirstymasStrat2
 - apdorojimas.tpp, 40
- skirstymasStrat3
 - apdorojimas.tpp, 40
- skirstymoPasirinkimas
 - main.tpp, 69
- start
 - Timer, 26
- strategijosPasirinkimas
 - isvestis.cpp, 45
 - isvestis.h, 50
- Studentas, 15
 - ~Studentas, 19
 - addNd, 19
 - egzaminoRezultatolvestis, 19
 - egzRandom, 19
 - galutinis, 19
 - genVardaPavarde, 20
 - getNd, 20
 - getRez, 20
 - namuDarbuRezultatulvestis, 20
 - nd, 24
 - ndRandom, 20
 - operator<<, 24
 - operator>>, 24
 - operator=, 21
 - read, 21
 - readRandom, 21
 - readSemiRandom, 22
 - readStudentConsole, 22
 - readStudentStream, 22
 - resizeNd, 23
 - rez, 24
 - setNd, 23
 - setRez, 23
 - Studentas, 18
 - studentoVardoPavardesIvestis, 23
- studentas.cpp
 - operator<<, 74

- operator>>, 74
- studentas.h
 - maxNdKiekis, 78
- studentoVardoPavardesIvestis
 - investis.h, 61
 - Studentas, 23
- studentuPasirinkimas
 - isvestis.cpp, 45
 - isvestis.h, 51
- TEST
 - gtest.cpp, 31, 32
- test.cpp
 - testas, 80
 - testCopyAssignmentOperator, 80
 - testCopyKonstruktorius, 80
 - testDefaultKonstruktorius, 80
 - testDestructor, 81
 - testInputOperator, 81
 - testMoveAssignmentOperator, 81
 - testMoveKonstruktorius, 81
 - testoRezultatas, 81
 - testOutputOperator, 82
 - testStreamKonstruktorius, 82
- test.h
 - testas, 84
 - testCopyAssignmentOperator, 84
 - testCopyKonstruktorius, 84
 - testDefaultKonstruktorius, 84
 - testDestructor, 85
 - testInputOperator, 85
 - testMoveAssignmentOperator, 85
 - testMoveKonstruktorius, 85
 - testoRezultatas, 85
 - testOutputOperator, 86
 - testStreamKonstruktorius, 86
- testas
 - test.cpp, 80
 - test.h, 84
- testavimoPasirinkimas
 - isvestis.cpp, 45
 - isvestis.h, 51
- testCopyAssignmentOperator
 - test.cpp, 80
 - test.h, 84
- testCopyKonstruktorius
 - test.cpp, 80
 - test.h, 84
- testDefaultKonstruktorius
 - test.cpp, 80
 - test.h, 84
- testDestructor
 - test.cpp, 81
 - test.h, 85
- testInputOperator
 - test.cpp, 81
 - test.h, 85
- testMoveAssignmentOperator
 - test.cpp, 81

- test.h, [85](#)
- testMoveKonstruktorius
 - test.cpp, [81](#)
 - test.h, [85](#)
- testoRezultatas
 - test.cpp, [81](#)
 - test.h, [85](#)
- testOutputOperator
 - test.cpp, [82](#)
 - test.h, [86](#)
- testStreamKonstruktorius
 - test.cpp, [82](#)
 - test.h, [86](#)
- Timer, [25](#)
 - durationDouble, [25](#)
 - elapsed, [26](#)
 - hrClock, [25](#)
 - reset, [26](#)
 - start, [26](#)
 - Timer, [26](#)
- vardas
 - Zmogus, [30](#)
- vector Directorijos aprašas, [13](#)
- vector/apdorojimas.cpp, [33](#), [34](#)
- vector/apdorojimas.h, [35](#), [37](#)
- vector/apdorojimas.tpp, [37](#), [41](#)
- vector/isvestis.cpp, [42](#), [46](#)
- vector/isvestis.h, [48](#), [52](#)
- vector/isvestis.tpp, [52](#), [54](#)
- vector/ivestis.cpp, [54](#), [56](#)
- vector/ivestis.h, [58](#), [62](#)
- vector/ivestis.tpp, [62](#), [63](#)
- vector/main.cpp, [63](#), [64](#)
- vector/main.h, [66](#), [67](#)
- vector/main.tpp, [68](#), [70](#)
- vector/random.cpp, [71](#), [72](#)
- vector/random.h, [72](#), [73](#)
- vector/studentas.cpp, [74](#)
- vector/studentas.h, [78](#)
- vector/test.cpp, [79](#), [82](#)
- vector/test.h, [83](#), [86](#)
- vector/Timer.h, [86](#), [87](#)
- vector/zmogus.cpp, [87](#)
- vector/zmogus.h, [88](#)

Zmogus, [27](#)

- ~Zmogus, [28](#)
- getPavarde, [29](#)
- getVardas, [29](#)
- operator=, [29](#)
- pavarde, [30](#)
- setPavarde, [29](#)
- setVardas, [29](#)
- vardas, [30](#)
- Zmogus, [28](#)